Quy gán RAG hai tầng

Shapley (tầng sinh) + Integrated Gradients trên đồ thị (tầng truy hồi)

Mục lục

1 Tổng quan **2**

2 Quy ước ký hiệu và khái niệm nền **4**
2.1 Lý thuyết trò chơi hợp tác — bảy khái niệm, một ví dụ . . . 4
2.2 Quy ước ký hiệu . . . 5

3 Tầng A — Tìm tập đủ tối tiểu **6**
3.1 Thiết lập . . . 6
3.2 Tập đủ tối tiểu — đối tượng cần tìm . . . 7
3.3 Ngưỡng và mốc tham chiếu . . . 9
3.4 Thu thập dữ liệu — bốn vòng . . . 10
3.5 Surrogate . . . 14
3.6 Liệt kê MWC . . . 16
3.7 Chỉ số cấu trúc . . . 17
3.8 Chọn S^* . . . 18
3.9 Credit và độ tin cậy . . . 18
3.10 Đầu ra . . . 18
3.11 Hai pha — mô hình ở pha 1, nhãn ở pha 2 . . . 19

4 Tầng B — Tìm đường trên đồ thị **25**
4.1 Điểm mục tiêu . . . 27
4.2 Gate trên cạnh . . . 28
4.3 Gate phải riêng từng tầng . . . 28
4.4 IG trên gate . . . 29
4.5 Tính chất chia đều — kiểm tra nội bộ miễn phí . . . 30
4.6 Bài toán: đọc ra chuỗi thông điệp . . . 31
4.7 Vẽ lại đồ thị: mỗi tầng một bản sao . . . 32
4.8 Quét một lượt (quy hoạch động) . . . 33
4.9 Trích đường — max-sum, không phải min-cost . . . 34
4.10 Kiểm chứng (bắt buộc) . . . 35
4.11 Hội tụ nhiều seed — greedy xóa cạnh, rồi tỉa ngược . . . 36
4.12 Nhiều đích — chia chung lượt forward . . . 40

5 Nhiều đích và quan hệ phụ thuộc **40**
5.1 Quan hệ này không bao giờ vòng . . . 40
5.2 Hai thứ rơi ra miễn phí từ bảng QHĐ . . . 41
5.3 Giải theo thứ tự nông → sâu . . . 41
5.4 Chỗ dễ sai: suff tính trên toàn đường . . . 41
5.5 Tiền tố có thể khác nhau giữa các bảng IG . . . 42
5.6 Đánh đổi . . . 42

6 Nối hai tầng và train **42**
6.1 Bảng chẩn đoán — chạy ở mỗi mốc . . . 42
6.2 Trọng số theo quan hệ phụ thuộc . . . 45
6.3 Ba hàm mất mát . . . 45
6.4 Bộ lọc cho giám sát đường đi . . . 47
6.5 Lịch đào nhãn và vòng lặp khởi động . . . 47
6.6 Vòng lặp ngược — dùng đích để tìm ứng viên mới . . . 49
6.7 Kỳ vọng thực tế . . . 50

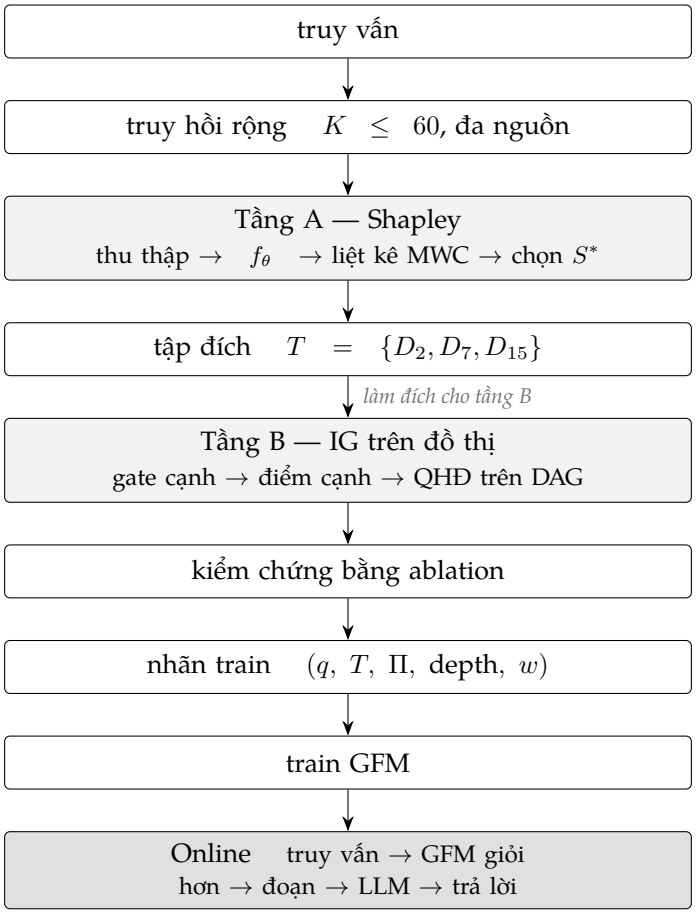
7 Chi phí **50**
7.1 Một truy vấn, pipeline đầy đủ . . . 50
7.2 Quy mô thật — hai pha . . . 50
7.3 Sổ dữ liệu . . . 51
7.4 So với bản Shapley . . . 51
7.5 Quy tắc chọn IG hay ablation . . . 51

8 Bất biến phải kiểm **52**

9 Thứ tự triển khai **53**

1 Tổng quan

	Câu hỏi	Công cụ	Đầu ra
Tầng A sinh	"đoạn nào thật sự tạo ra câu trả lời?"	Shapley + greedy	tập đích T
Tầng B truy hồi	"đáng lẽ phải đi lối nào tới đó?"	IG trên gate cạnh + QHĐ trên DAG	cây đường đi Π



Vì sao cần cả hai. Shapley là công cụ duy nhất xử lý đúng *dư thừa* (hai cụm trùng ý thì chia nhau công) và *bổ sung* (hai cụm chỉ có giá trị khi đi cùng — chính là multi-hop). IG có tính chất riêng: chi phí của nó không phụ thuộc số đơn vị cần chấm, nên nó là lựa chọn duy nhất chạy nổi trên hàng nghìn cạnh đồ thị.

Vai của graph foundation model.

LLM hàng tỷ tham số	biết ngôn ngữ + tri thức	→ tầng A
GFM 34M	biết *cách đi*	→ tầng B
Đồ thị dữ liệu	chứa tri thức cụ thể	→ thay trí nhớ tham số

GFM không nhớ thực thể nào. Khởi tạo của nó là $h_v^{(0)} = \text{MLP}(\text{emb}(v), \text{emb}(q))$ cho seed và $\text{emb}(v)$ cho phần còn lại — $\text{emb}(v)$ là embedding văn bản, không phải tham số gắn với ID. Nhờ vậy nó chạy được trên đồ thị mới, thực thể mới, không cần train lại. Đó là nghĩa của chữ "foundation" ở đây.

Hệ quả: 34M tham số là đủ, nên tầng B gần như miễn phí so với tầng A — và IG chạy nổi trên hàng nghìn cạnh.

Phạm vi đóng góp

Tài liệu này là thiết kế, chưa có kết quả thực nghiệm. Bảng dưới tách rõ cái gì lấy từ literature, cái gì là đề xuất.

Từ literature	Đề xuất ở đây
Shapley value [1]	bản thay thế, đã tách sang tài liệu riêng
Deegan–Packel, PGI [4, 6]	credit chia đều trong tập đủ tối tiểu (§3.9)
Integrated Gradients [2]	gắn vào *gate cạnh* của GNN thay vì embedding đầu vào (§4.4)
Mở rộng đa tuyến tính [3]	dùng tính chất c/h làm phép kiểm nội bộ (Kiểm tra 5, §4.5)
Banzhaf [5]	bản có dấu: $\beta < 0$ bắt đoạn đẩy ngược (§3.7)
PDist / LOOP của Atlas [7, 8]	thay target bằng tập đủ tối tiểu; thêm $w(D)$ cho đích phụ thuộc nhau (§6.2)
GFM-RAG, G-reasoner [9, 10]	chèn gate phụ thuộc truy vấn (§4.2 chế độ 2); $\mathcal{L}_{\text{hop}}$, $\mathcal{L}_{\text{edge}}$ (§6.3)
Mask cạnh giải thích GNN [11, 12]	gate riêng *từng tầng*, không phải một gate chung mỗi cạnh (§4.3)
Thuần đề xuất: max-sum trên DAG trải phẳng thay min-cost (§4.9); greedy xóa cạnh + tỉa ngược cho hội tụ (§4.11); thứ tự theo depth (§5.3); vòng lặp ngược $R(T) \setminus R(V_q)$ (§6.6).	

Vị trí so với Atlas. Atlas [8] đưa ra ba cách lấy tín hiệu từ LM để train retriever. Nhìn qua lăng kính Shapley thì chúng là ba *vị trí* khác nhau của cùng một đại lượng:

```
PDist     log p(y | q, d_k)                marginal khi d_k đến đầu tiên
LOOP      v(tất cả) - v(tất cả trừ d_k)    marginal khi d_k đến cuối cùng
Shapley   trung bình trên mọi vị trí
```

Atlas chọn PDist vì ổn định hơn EMDR^2/ADist và rẻ hơn LOOP. Tài liệu này lập luận rằng với bằng chứng dư thừa thì LOOP tụt về 0, còn với multi-hop thì PDist tụt về 0 — cả hai đầu mút đều lệch, mỗi cái lệch ở một loại truy vấn. Đề xuất ở đây là trả giá cho phần *trung bình*, nhưng gom cụm trước để chi phí còn chịu được.

2 Quy ước ký hiệu và khái niệm nền

2.1 Lý thuyết trò chơi hợp tác — bảy khái niệm, một ví dụ

Tài liệu mượn công cụ từ lý thuyết trò chơi hợp tác. Mục này giải thích đủ để đọc tiếp, không giả định người đọc biết trước gì cả.

```
người chơi      một đoạn văn trong pool
liên minh S     một tập đoạn.  n đoạn thì có 2^n liên minh
hàm đặc trưng   v(S) = liên minh này làm được gì
                     = chất lượng câu trả lời khi chỉ đưa S cho model
```

Hai dòng cuối là toàn bộ cầu nối giữa RAG và lý thuyết trò chơi: **đoạn văn là người chơi, chất lượng câu trả lời là tiền thưởng.**

Từ trò chơi có giá trị sang trò chơi thắng/thua. $v(S)$ là số thực. Đặt một ngưỡng τ thì nó thành *trò chơi đơn giản* — chỉ còn thắng hoặc thua: S thắng $\iff v(S) \geq \tau$. Và khái niệm trung tâm của cả tầng A là **liên minh thắng tối tiểu** (minimal winning coalition, MWC): thắng, mà bỏ bất kỳ thành viên nào cũng thành thua.

Một ví dụ chạy qua hết. Ba đoạn a, b, c, giả sử:

```
v(rỗng) thua    v(a)  thua     v(b)  thua     v(c)   thua
v(ab) thắng     v(ac) thắng    v(bc) thua     v(abc) thắng

MWC:   { a,b }  và  { a,c }
       {a,b,c} thắng nhưng không tối tiểu -- bỏ c vẫn thắng
```

Từ đó đọc ra ba chỉ số, và cả ba đều nói cùng một điều: a **là đoạn then chốt.**

```
PGI_i          tỉ lệ MWC có chứa i          (Public Good Index)
   PGI(a) = 2/2 = 1      PGI(b) = PGI(c) = 1/2

Deegan-Packel  chia đều trong mỗi MWC, rồi trung bình trên các MWC
   a:  1/2*(1/2 + 1/2) = 1/2     b:  1/2*1/2 = 1/4     c:  1/4

swing          thêm i vào S làm S từ thua thành thắng
Banzhaf beta_i tỉ lệ swing trên mọi S không chứa i
   a:  {b}->{ab} có   {c}->{ac} có   {b,c}->{abc} có   rỗng->{a} không
       beta_a = 3/4
   b:  {a}->{ab} có,  còn lại không                ->  beta_b = 1/4
   c:  đối xứng với b                              ->  beta_c = 1/4
```

Shapley, và vì sao tài liệu này không dùng nó. Giá trị Shapley ϕ_i là đóng góp biên *trung bình* của i trên *mọi* thứ tự xuất hiện, và thỏa tiên đề đầy đủ $\sum_i \phi_i = v(\text{toàn bộ}) - v(\emptyset)$. Nó trả lời "chia công thế nào" — công bằng theo tiên đề, nhưng cần duyệt 2^n liên minh, với $n = 40$ là bất khả thi. Mà câu ta thật sự cần không phải chia, là **chọn**: *tập nào là đủ*. Đó là lý do tài liệu chuyển sang MWC (§3) và giữ bản Shapley ở tài liệu riêng.

Hai khái niệm phụ, dùng ở §3.8. *Trội Pareto*: A trội B nếu A tốt hơn hoặc bằng B ở mọi tiêu chí và tốt hơn hẳn ở ít nhất một; B bị trội thì loại thẳng. *Đơn điệu*: thêm người chơi không bao giờ làm liên minh thắng thành thua — lý thuyết chuẩn hay giả định điều này, còn ta thì *không*, vì đoạn gây nhiễu làm v giảm (§3.2).

Và một khái niệm từ phía quy gán, dùng cho tầng B. *Tiên đề đầy đủ* (completeness): tổng các phần quy gán đúng bằng hiệu giữa "có tất cả" và "không có gì"; *baseline* là trạng thái "không có gì" — tầng A là xóa mọi đoạn, tầng B là tắt mọi gate. Completeness là thứ làm mọi con số đọc được như "chiếm bao nhiêu phần trăm", và là nền của mọi phép kiểm trong §8.

2.2 Quy ước ký hiệu

Mỗi mục lớn mở đầu bằng một bảng nhỏ chỉ liệt kê ký hiệu mục đó giới thiệu. Ở đây chỉ nêu quy ước chung và ba chỗ dễ nhầm.

```
chỉ số trên  ^(l)     luôn là tầng GNN            l = 0..L
chỉ số dưới  _k       luôn là chỉ số đoạn         k = 1..K
             _c       chỉ số cụm                  c = 1..C
             _e       cạnh đồ thị
dấu mũ       ghat     đại lượng đã học hoặc đã chuẩn hóa
dấu phẩy     E'       baseline -- trạng thái "đã tắt"
```

T vs n	T là **tập đích** (§3.8); số token là n (§3).
E vs $\mathcal{E}$	E là ma trận embedding (§3); $\mathcal{E}$ là tập cạnh đồ thị (§4). Hai nghĩa không bao giờ xuất hiện trong cùng một công thức.
h vs h_v	h trần là số hop của một đường (§4.5); h có chỉ số dưới (h_v, h_r, h_q) là embedding (§4.2).

3 Tầng A — Tìm tập đủ tối tiểu

Bài toán. Model đã trả lời xong với K đoạn truy hồi. Đoạn nào *thật sự* tạo ra câu trả lời đó?

Mục tiêu. Không phải chia công cho từng đoạn, mà tìm các **tập đoạn đủ và tối tiểu**: bỏ bất kỳ phần tử nào thì không còn đủ.

Vì sao đây mới là câu hỏi đúng. Shapley trả lời "mỗi đoạn đóng góp bao nhiêu" — bài toán *chia*, và nó cần 2^K liên minh. Câu ta thật sự cần là "tập nào là đủ" — bài toán *chọn*. Đổi đối tượng thì đổi luôn cả chi phí lẫn cách đọc kết quả.

```
Shapley   phi_i thuộc R cho mọi đoạn   một số mỗi đoạn, cộng lại bằng cái bánh
MWC       W = { S_1, ..., S_m }        nhiều tập, mỗi tập tự nó là một lời giải thích
```

Bản Shapley vẫn còn trong tài liệu riêng *Tầng A bản Shapley* — nó sinh dữ liệu huấn luyện cho §3.5 và là phương án lui.

Ký hiệu mục này giới thiệu.

```
K, C              số đơn vị trong pool (<= 60); tập ứng viên { c_1, ..., c_n }
n                 |C| sau rerank, 10-30
r_i               điểm rerank của đơn vị i -- chỉ dùng để phá hòa ở §3.8
v(S)              điểm chất lượng câu trả lời khi chỉ đưa S
tau_q             ngưỡng đủ của truy vấn q (§3.3)
W                 tập các MWC tìm được
f_theta(q,S)      surrogate -- xấp xỉ 1[ v(S) >= tau_q ]
beta_i, PGI_i     Banzhaf có dấu; số MWC chứa i
S*                MWC được chọn để báo cáo
C, m              số cụm; số ứng viên được chấm mỗi vòng greedy (§3.4)
mảnh              sub-claim -- câu hỏi chẻ thành các khẳng định 1-hop (§3.2)
adq(S)            S có phủ hết mảnh không -- CÓ BẰNG CHỨNG, khác với v (§3.2)
bảng phủ          mảnh nào được đoạn nào entail; tính 0 lượt LLM (§3.2)
r                 SỐ VÒNG greedy -- KẾT QUẢ, không phải núm vặn:
                  r = |S| lúc greedy dừng, tức cỡ tập đủ nó tìm được
spread            max_k v({k}) - v(rỗng) -- singleton có tín hiệu hay không
```

3.1 Thiết lập

```
sinh    a^_S   =  generate( q, S )                        ~80 token
chấm    v(S)   =  P_judge( "đúng" | q, a^_S, y )  trong [0,1]
                  đọc logprob token đầu, không decode câu trả lời judge
v(rỗng)        =  điểm khi không có đoạn nào -- chỉ tri thức tham số
```

Vì sao không dùng log-likelihood của đáp án vàng. Nó rẻ hơn nhưng đo sai thứ: model trả lời "Paris" trong khi $y =$ "Paris, France" là *đúng về ngữ nghĩa* mà $\log P(y)$ lại thấp. Sai lệch đó không triệt tiêu khi so giữa các tập con — ca xấu là đoạn tốt làm model trả lời dứt khoát (gold dài hơn nên điểm thấp), còn đoạn mơ hồ làm nó vòng vo và vô tình khớp gold hơn. Thứ tự lật.

Vì sao judge phải trả số liên tục, không phải thang 1–5. Judge rời rạc cho v chỉ 5 mức, kéo theo hàng chục tập con hòa nhau, không phân biệt được S với $S \setminus \{c\}$ — "tối tiểu" mất nghĩa và greedy không tìm được chỗ cắt τ_q. Nên hỏi judge một câu yes/no rồi **đọc xác suất của token đầu**, chứ đừng bắt nó chấm điểm.

Chi phí so với log-likelihood: khoảng $+12\%$, không phải vài chục lần. Với $n = 20$ chunk $\times$ 250 token: log-likelihood tốn ≈ 5.530 token quy đổi, còn sinh + judge tốn ≈ 6.220 (prompt 5.500, sinh 80 token tính hệ số 4, lượt judge 400). Chi phí bị chi phối bởi việc nạp prompt mà *cả hai* đều phải trả. Nhờ vậy ngân sách theo từng truy vấn ở §3.4 vẫn giữ được, và f_θ **không bị ép phải tổng quát hóa qua truy vấn** — nó chỉ cần khớp trong một truy vấn.

Phân tầng (tùy chọn). Dùng $\log P(y \mid q, S)$ làm proxy cho vòng 0–2 và chỉ chạy judge ở vòng 3. Hiệu chuẩn bằng $\rho = \text{Spearman}(v_{\text{cheap}}, v_{\text{judge}})$ trên mẫu 100 tập con: $\rho \geq 0{,}7$ thì giữ, $\rho < 0{,}7$ thì bỏ proxy và chạy judge cho cả vòng 0–2 — chỉ đắt thêm 12%. Chỉ số theo dõi là tỷ lệ S^* bị vòng 3 bác. Phân tầng là tối ưu hóa *tùy chọn*, không phải điều kiện sống còn.

Cần có đáp án vàng y, nên cách này dành cho khâu *đào nhãn offline*.

Đơn vị của pool, và hai con số đừng lẫn.

```
K <= 60    pool tho, da nguon, sau khu trung lap
   | rerank theo từng nguồn
n  = 40    20 BM25  +  15 GFM  +  5 ngau nhien     <- day moi la thu vao §3.4
```

Chỉ **hai** trong bốn loại node của QuadGraph vào pool; `entity` và `attribute` không phải đơn vị văn bản nên không bao giờ vào prompt — vai trò của chúng ở §6.3.

Rerank theo từng nguồn, không gộp chung. Pool đa nguồn sinh ra để bắt thứ GFM đang bỏ sót; nhưng nếu reranker là cross-encoder huấn luyện trên cùng loại dữ liệu với retriever thì nó **chia sẻ điểm mù** — đoạn mà BM25 lôi về đúng vì GFM không thấy sẽ bị nó loại ngay, và ô "lỗi đồ thị" của bảng chẩn đoán lại không bao giờ khác rỗng. Giữ hạn ngạch riêng cho mỗi nguồn thì không nguồn nào bị xóa sổ. Ba ràng buộc về đơn vị:

```
cỡ chunk     200-300 token, cắt theo ranh giới đoạn văn / mục, không cắt giữa câu
             quá dài -> quy gán thô; quá ngắn -> một sự kiện bị cắt đôi và sinh ra
             chữ ký multi-hop giả (hai nửa đều vô dụng khi đứng riêng)

không chồng lấp   chunk chồng nhau tạo ra dư thừa nhân tạo. Nếu khâu truy hồi cần
             chồng lấp thì khử trùng lặp trước khi lập pool.

chung một cách chunk   pool và đồ thị phải dùng cùng một cách cắt (tiền kiểm §6.1).
```

Pool lấy từ **nhiều nguồn** — BM25 + GFM + ngẫu nhiên. Lý do ở §6.6: nếu pool chỉ từ GFM thì nhãn không bao giờ chứa thứ model đang bỏ sót, và ô "lỗi đồ thị" của bảng chẩn đoán không bao giờ có thể khác rỗng.

3.2 Tập đủ tối tiểu — đối tượng cần tìm

```
S đủ         v(S) >= tau_q        VÀ     adq(S) = phủ hết
S tối tiểu   đủ,  và không tập con thật sự nào của S còn đủ

W  =  { mọi S đủ tối tiểu }
```

«Đủ» có hai thành phần, và v chỉ đo một. Các bản trước của tài liệu này định nghĩa S đủ $\iff v(S) \geq \tau_q$. Điều đó **sai**, và sai theo hướng sinh ra nhãn quy gán giả.

```
v(S)      model đọc S rồi trả lời -- câu đó có khớp y không?     ->  TRẢ LỜI ĐÚNG
adq(S)    các đoạn trong S có THẬT SỰ nói ra đáp án không?       ->  CÓ BẰNG CHỨNG
```

Hai thứ này lệch nhau, vì model có thể trả lời đúng **nhờ nhớ sẵn**:

```
q = "ai đạo diễn phim thắng Oscar phim hay nhất 1994?"     y = "Robert Zemeckis"

đoạn C = "Forrest Gump là một hiện tượng văn hóa thập niên 1990."
         -> không nói ai đạo diễn, cũng không nói thắng giải nào

model đọc C -> thấy "Forrest Gump" -> tự nhớ ra Zemeckis -> trả lời ĐÚNG
->  v({C}) cao  ->  định nghĩa cũ nhận C vào T
->  rồi đem nhãn đó dạy retriever đi lấy C.   NHÃN SAI.
```

Phép trừ $v(\emptyset)$ không cứu được: q trần không gợi ra ký ức, còn chữ "Forrest Gump" trong C thì gợi ra, nên $v(\{C\}) \gg v(\emptyset)$ thật — phép trừ đang **tính công cho cái mồi**.

Đây không phải ca giả định. Joren et al. (ICLR 2025) đo được trên HotpotQA rằng model trả lời *đúng* 35–62% số ca mà ngữ cảnh *không đủ*; tỷ lệ trả lời sai tăng từ $10{,}2\%$ (không ngữ cảnh) lên $66{,}1\%$ (ngữ cảnh không đủ). Và tỷ lệ ca có ngữ cảnh đủ sau truy hồi: **MuSiQue-Ans** $46{,}2\%$**, HotpotQA** $44{,}6\%$ — tức hơn nửa số truy vấn trên đúng hai bộ §3.1 nhắm tới *không có* tập đủ trong pool.

***adq* đo thế nào — chẻ câu hỏi, rồi đối chiếu.** Định nghĩa "đủ ngữ cảnh" của Joren et al. *không* tham chiếu đáp án vàng, nên bài của họ khó (autorater 93%, RINSE $\sim 84\%$). Tầng A chạy **offline, có** y, nên hỏi được câu dễ hơn: *"các đoạn này có chống đỡ cho y không?"* — tức **kéo theo**, không phải suy đoán.

```
bước 1   CHẺ câu hỏi thành mảnh    1 lượt LLM, MỖI TRUY VẤN (không phải mỗi tập con)
         vào  q + y
         ra   mảnh 1  "phim thắng Oscar phim hay nhất 1994 là Forrest Gump"
              mảnh 2  "Forrest Gump do Robert Zemeckis đạo diễn"

bước 2   ĐỐI CHIẾU mỗi đoạn x mỗi mảnh    model NLI nhỏ, local, 0 LƯỢT API
         ra   entail / neutral / contradict
```

Phải chẻ ra, vì không chẻ thì *không chạy được*, hai lý do độc lập:

```
ĐỘ DÀI    premise = cả S có thể tới 10.000 token, model NLI chỉ nhận 512.
          chẻ ra thì mỗi lượt kiểm là 1 đoạn (250 token) đối 1 câu ngắn.

BẮC CẦU   claim ghép "phim thắng 1994 do Zemeckis đạo diễn" thì KHÔNG đoạn nào
          một mình khẳng định được -> loại oan cả {A,B}, tức loại đúng ca multi-hop.
          chẻ ra thì mỗi mảnh có đúng một đoạn khẳng định nó.
```

Điểm thứ hai là phát hiện trung tâm của arXiv:2608.00585: kiểm từng đoạn đối câu hỏi gốc thì thất bại ở multi-hop, và phân rã sửa được. (Tài liệu này lấy *cơ chế* từ đó; chưa trích số vì chưa đọc được bản text.)

Bảng phủ — thứ mọi mục sau đây dùng tới. Kết quả bước 2 là một bảng, tính xong với **0 lượt LLM**:

```
                            mảnh 1              mảnh 2
đoạn  3                     entail                 --
đoạn  7                     entail                 --
đoạn 12                       --                entail
đoạn 23                       --                entail
đoạn 31                       --                entail
28 đoạn còn lại               --                   --

adq(S) = phủ hết  <=>  mọi mảnh có ít nhất một đoạn trong S entail nó
```

Bảng này cho ba thứ, cả ba miễn phí:

```
1  CỬA THOÁT    có mảnh nào không đoạn nào chống đỡ -> pool không trả lời được
                -> bỏ truy vấn trước khi tiêu đồng nào   (§3.11 AO/CO)
2  ỨNG VIÊN     bài toán phủ tập nhỏ, giải CHÍNH XÁC:
                   mảnh 1 <- {3,7}   mảnh 2 <- {12,23,31}
                   nghiệm:  {3,12} {3,23} {3,31} {7,12} {7,23} {7,31}
                -> dùng làm điểm khởi cho §3.4 và §3.6
3  PRIOR |T|    số mảnh nói |T| NÊN bằng bao nhiêu.
                |S*| = 8 mà 2 mảnh -> phình, chắc chắn
```

Vì sao vẫn cần cả hai, không bỏ cái nào.

```
                     adq phủ hết                    adq KHÔNG phủ
v(S) >= tau_q   nhãn TỐT -- nhận              MỒI THAM SỐ -- loại
                                              (lỗ mà định nghĩa cũ không thấy)
v(S) <  tau_q   bằng chứng CÓ mà model        pool THIẾU -- thoát sớm ở AO/CO
                không nối được
                -> lỗi MODEL SINH, đáng báo cáo
```

Không mệnh đề nào suy ra mệnh đề kia. adq giữ **tách khỏi** v, không gộp thành một điểm số — gộp thì mất bảng 2×2 này, và mất luôn khả năng tách "lỗi truy hồi" khỏi "lỗi model sinh". Ô dưới-trái là ô mới mà tầng A trước đây không tách được: bằng chứng đủ mà model vẫn sai thì **tầng B không sửa được** — phải đổi model sinh hoặc đổi prompt.

Không cần v đơn điệu. Lý thuyết simple game thường đòi đơn điệu, mà v của ta thì không — thêm một đoạn gây nhiễu làm v *giảm*. Nhưng định nghĩa trên không dùng tới đơn điệu ở đâu cả. Và nếu có ai muốn cưỡng bức đơn điệu bằng bao đơn điệu $\bar{v}(S) = \max_{S' \subseteq S} v(S')$ thì cũng vô ích: S thắng tối tiểu của $\bar{v}$ kéo theo S thắng nhờ một $S' \subseteq S$; nếu $S' \subsetneq S$ thì S' cũng thắng, trái tính tối tiểu; vậy $S' = S$, tức $v(S) \geq \tau_q$. Hai bên trùng khít: $\mathrm{MWC}(\bar{v}) = W$. Nên đơn điệu hóa không đổi gì ngoài chi phí.

Dư thừa hiện ra thành bội số MWC. Ba đoạn trùng ý sinh ra ba MWC khác nhau, mỗi tập chứa một đoạn. Đây là lý do bản này *không cần* gom cụm để hạ 2^K: dư thừa tự lộ trong cấu trúc W, còn 2^K thì surrogate lo.

3.3 Ngưỡng và mốc tham chiếu

$$\tau_q = \theta \cdot \big[v_{\text{ref}}(q) - v(\emptyset)\big] + v(\emptyset), \qquad \theta = 0{,}95$$

$$v_{\text{ref}}(q) = \max\big(v(C),\ \max v(S) \text{ đã quan sát}\big)$$

Vẫn trừ $v(\emptyset)$, dù v giờ nằm trong $[0,1]$ chứ không còn âm. Với log xác suất thì phép trừ là *bắt buộc*; với judge thì nó thành *lựa chọn*, và ta giữ vì nó đo đúng thứ cần — phần mà truy hồi *đóng góp thêm*, chứ không phải độ đúng tuyệt đối. Không trừ thì với $v(\emptyset) = 0{,}60$ và $v(C) = 0{,}95$, ngưỡng là $0{,}90$ và một đoạn yếu cũng "đủ"; có trừ thì ngưỡng là $0{,}93$, đòi lấy lại 95% phần truy hồi thật sự thêm vào.

Chốt chặn khi mẫu số nhỏ. Nếu $v_{\text{ref}} - v(\emptyset) < \delta$ ($\delta \approx 0{,}05$) thì truy hồi gần như không đóng góp gì — đừng chia cho một số gần 0, mà báo $T = \emptyset$ kèm cờ "truy hồi không đóng góp". Đây là ca có thật và đáng báo cáo, không phải lỗi.

Và mốc phải là **cực đại quan sát được**, không phải $v(C)$: khi trong C còn đoạn gây nhiễu thì một tập con có thể ghi điểm cao hơn cả C, và neo vào $v(C)$ là neo vào một mốc bị dìm. $v(C)$ tốn một lượt; phần max thì miễn phí vì §3.4 vốn đã đánh giá hàng chục tập con.

3.4 Thu thập dữ liệu — bốn vòng

Mục này quyết định cả phương án, nên nói rõ f_θ **cần học cái gì** trước. §3.6 dùng f_θ để làm đúng một việc: hỏi $f_\theta(S) \geq \tau_q$? rồi tỉa về tối tiểu. Nó không bao giờ cần giá trị $v(S)$ chính xác ở chỗ khác.

```
không phải   hồi quy chính xác v(S) trên toàn không gian
mà là        phân loại đúng dấu của ( v(S) - tau_q ), sát biên
             + thứ tự thô đủ tốt cho beam chọn nhánh
```

Tức mục tiêu là **bộ phân loại ở biên**. Hai kiểu lấy mẫu phổ biến đều sai hướng: lấy mọi cặp đôi tốn 190 lượt mỗi truy vấn với $n = 20$ mà gần như toàn negative *dễ*; lấy tập con ngẫu nhiên thì phần lớn rơi xa ngưỡng. Với |MWC| điển hình 3–5 thì cặp đôi hầu hết thiếu, và thiếu rất xa ngưỡng.

Vòng 0 — khung bắt buộc. $v(\emptyset)$, $v(C)$ (cho τ_q), và mọi singleton: biết đoạn nào tự đủ một mình, và cho f_θ tín hiệu bậc 1. Tổng $n + 2 \approx 22$ lượt.

Vòng 1 — đi dọc biên bằng greedy. Greedy từ rỗng, mỗi vòng chọn thêm một đoạn, tới khi vượt τ_q. Ghi lại *mọi* lượt chấm, cả cái bị loại: cái được chọn là mẫu ở biên phía trên, cái bị loại là mẫu ở biên phía dưới — và cái thứ hai mới là thứ quý.

Không quét hết n ứng viên mỗi vòng. Quét đầy đủ tốn $rn - r(r-1)/2$ lượt — với $n = 40, r = 3$ là 117, tăng tuyến tính theo n. Thay vào đó gom cụm rồi chỉ thăm dò đại diện:

Gom pool thành C cụm, shortlist mỗi vòng m ứng viên thay vì cả 40.

C có trần cứng, không phải hằng số cho sẵn. Mỗi cụm được một **sàn** 1 lượt, nên sàn tiêu hết C trong ngân sách m. Nếu $C = m$ thì sàn ăn trọn ngân sách, phần bốc theo xác suất biến mất, và phân bổ thoái hóa thành chia đều — mất đúng cơ chế "cụm nóng được nhiều hơn" mà mục dưới dựng lên. Vậy

$$C \;\leq\; \lfloor m/2 \rfloor \qquad m = 12 \Rightarrow C \leq 6; \quad m = 30 \Rightarrow C \leq 15$$

Con số $C = 6$ dùng trong các ví dụ dưới là *trần ở* $m = 12$, không phải lựa chọn tùy ý. Trong trần đó thì dữ liệu quyết:

```
pha 1   C = argmax silhouette trên embedding pool,  với  2 <= C <= floor(m/2)
pha 2   gom theo PHẦN DƯ của g_theta, không theo cosine embedding:
            d(k,k')  =  g_theta(q,{k}) + g_theta(q,{k'}) - g_theta(q,{k,k'})
            d lon   -> trùng lặp -> CÙNG cụm
```

```
           d âm    -> bắc cầu   -> KHÁC cụm
      780 cặp cho 40 chunk, mỗi cặp một lượt forward mô hình nhỏ. 0 lượt LLM.
```

Mục đích gom cụm là nhóm các chunk *thay thế được cho nhau* lại, để sàn-mỗi-cụm bảo đảm shortlist chạm nhiều chủ đề. Cosine embedding chỉ là proxy cho tính thay thế đó; sau pha 1 (§3.11) ta có thứ đo trực tiếp nó, nên nên dùng. Gom cụm ở đây *không* phải để hạ 2^K: nó để shortlist có **tính đa dạng**, vì lấy top-12 theo prior có thể vớ phải 12 bản sao của nhau.

Greedy đi lên có ràng buộc phủ. Bảng phủ (§3.2) đổi ba chỗ trong greedy, và một trong ba làm nó *rẻ hơn*.

```
HƯỚNG ĐI   trước   chọn đoạn làm v tăng nhiều nhất
           sau     trong các đoạn phủ được một mảnh CHƯA PHỦ,
                   chọn đoạn làm v tăng nhiều nhất
                   -> phủ là ràng buộc CỨNG, v là mục tiêu trong ràng buộc

DỪNG       trước   v(S) >= tau_q
           sau     v(S) >= tau_q  VÀ  S phủ hết mảnh
                   v >= tau_q mà chưa phủ hết = ĐÚNG ca mồi tham số -> đi tiếp

pi_c       trước   max gain ước từ singleton -- mù ở multi-hop (xem mục dưới)
           sau     ưu tiên cụm chứa đoạn chống đỡ mảnh CÒN CHƯA PHỦ
```

Chỗ π_c là chỗ đáng tiền. Mục dưới chỉ ra rằng ở multi-hop *mọi* singleton $\approx v(\emptyset)$ nên π_c là nhiễu, và đó là lý do m phải nâng lên 30. Bảng phủ nói đoạn nào mang bằng chứng **kể cả khi không singleton nào làm v nhích**:

```
trước   multi-hop: spread thấp -> m = 30 -> 67.500 token greedy
sau     prior không mù nữa      -> m = 12 -> 27.000 token
                                   tiết kiệm 40.500 token mỗi truy vấn multi-hop
```

Nên mục «m phải thích nghi» dưới đây vẫn đúng về *lý do* (prior mù thì phải quét rộng), nhưng điều kiện kích hoạt đổi: $m = 30$ chỉ cần khi `spread` thấp **và** bảng phủ cũng không cho tín hiệu — tức khi phân rã thất bại.

r là kết quả, không phải núm vặn. Greedy thêm một đoạn mỗi vòng tới khi vượt τ_q, nên $r = |S|$ lúc dừng — tức r chính là cỡ tập đủ mà greedy tìm được. Câu 2-hop cho $r = 2$, 3-hop cho $r = 3$; con số $r = 3$ ở phần ngân sách là ước lượng để tính tiền, không phải cấu hình. Sau khi tỉa ở §3.6 thì $|S^*|$ có thể còn nhỏ hơn r. Và **vòng greedy đầu tiên miễn phí** — nó chính là chấm singleton mà vòng 0 đã làm, nên chi phí là $(r-1) \cdot m$ chứ không phải $r \cdot m$.

Chia m lượt cho các cụm thế nào. Chia đều $a = m/C$ là cứng nhắc theo đúng hai chiều: cụm vô dụng vẫn ngốn đủ phần của nó, còn cụm chứa ba ý quan trọng thì bị bóp. Thay bằng **sàn cứng cộng phần còn lại bốc theo xác suất**:

```
tiềm năng của cụm c tại vòng t:

    pi_c  =  max_{k thuộc c \ S_t}  gain^( k | S_t )

    gain^   vòng 2:   v({k}) - v(rỗng)                dùng singleton đã có
            vòng 3+:  f_theta(S_t + {k}) - f_theta(S_t)

phân bổ:
```

```
a_c  =  1  +  Multinomial( m - C ;  softmax(pi_c / T) )_c
        ---               ---------------------------
        sàn, mọi cụm      phần dư chia theo xác suất
```

Với $C = 6, m = 12$: sàn tiêu 6, còn 6 lượt bốc theo π. Cụm nóng có thể nhận $1 + 4 = 5$, cụm chết chỉ nhận 1. Tổng vẫn đúng m — chỉ đổi cách chia, không đổi ngân sách.

m phải thích nghi, vì prior mù đúng ở ca ta quan tâm. Ở vòng 2, π_c lấy từ $v(\{k\}) - v(\emptyset)$. Với single-hop thì một đoạn tự nó gần đủ nên singleton có tín hiệu rõ; với *multi-hop* thì mọi đoạn đứng riêng đều $\approx v(\emptyset)$ và prior thành nhiễu — mà multi-hop chính là lý do tồn tại của cả thiết kế. Khi đó chọn 12 trên 40 gần như bốc ngẫu nhiên: đoạn cần nằm trong một cụm 7 phần tử, xác suất lọt vào 2 lượt thăm dò chỉ ≈29%. Và m nhỏ *chưa chắc rẻ hơn*: shortlist trượt thì r tăng, chi phí chuyển từ "vòng rộng" sang "nhiều vòng" chứ không biến mất.

Ta đã đo sẵn thứ cần để quyết, ngay ở vòng 0:

```
spread  =  max_k v({k}) - v(rỗng)

spread >= 0.15 * [ v_ref - v(rỗng) ]   prior dùng được      ->  m = 12
spread <  ngưỡng đó                    prior mù, multi-hop  ->  m = 30

m      lượt    token greedy    tổng mỗi truy vấn
12      24        27.000          124.000
20      40        45.000          142.000
30      60        67.500          164.000
40      80        90.000          187.000   (quét đầy đủ, không gom cụm)
```

Nếu một nửa tập là multi-hop thì trung bình ≈143.000 token, tức $+15\%$.

Ô "khám phá" nên đo độ phủ truy vấn, không phải khoảng cách tới S. Khi singleton phẳng vẫn còn một tín hiệu rẻ chưa dùng — phần truy vấn mà S hiện tại chưa phủ: $\pi_c = \max_{k \in c \setminus S} \text{dophu}\big(q \ominus S,\ k\big)$. Thuần embedding, không tốn lượt LLM nào, và nó có tín hiệu *đúng lúc* singleton không có, vì multi-hop nghĩa là mỗi đoạn phủ một mảnh khác nhau của câu hỏi. Khoảng cách tới S chỉ nói "khác S"; độ phủ truy vấn nói "bù đúng chỗ S còn thiếu".

Vì sao bốc xác suất chứ không chia tỉ lệ tất định. Chia tất định thì một cụm bị chấm thấp ở vòng 2 sẽ *vĩnh viễn* không được thăm lại — khóa chặt sai lầm sớm. Bốc xác suất giữ cho thăm dò nằm sẵn trong cơ chế. Đây là tinh thần Thompson sampling mà không phải dựng hẳn một bandit; với $r \approx 3$ vòng thì bandit đầy đủ chưa kịp học gì.

Sàn $= 1$ là lưới an toàn cho khâu gom cụm, không phải cho công bằng: nếu gom cụm nhầm hai sự kiện vào một chỗ mà cụm đó bị đánh giá thấp, không có sàn thì nó bị bỏ hẳn và mất luôn một đoạn cần thiết. Trong một cụm, $a_c = 1$ thì lấy ứng viên khai thác (π cao nhất), $a_c \geq 2$ thì 1 khai thác cộng $a_c - 1$ khám phá (xa S_t nhất theo embedding).

Độ phủ tích lũy qua các vòng. Ứng viên "khám phá" đổi theo S, mà S đổi mỗi vòng — nên qua 3 vòng, một cụm 7 phần tử được thăm dò tới 6 phần tử khác nhau. Nhiệt độ T là núm duy nhất thêm vào và nó thay cho a: $T \to 0$ dồn hết vào cụm nóng nhất, $T \to \infty$ chia đều tức quay về a cố định.

Đường greedy **cắt ngang τ_q đúng một lần**, và các ứng viên bị loại ở vòng cuối là những tập *suýt đủ*. Không có cách lấy mẫu nào dày ở biên hơn thế, và nó rơi ra tự nhiên chứ không phải dò mò. Đây cũng là lý do bản Shapley cũ vẫn có ích: vòng greedy của nó sinh ra đúng loại dữ liệu này.

Vòng 2 — active learning. Train $k = 5$ surrogate trên bootstrap của vòng 0–1, rồi chọn mẫu thỏa *cả hai*: ensemble bất đồng cao, và $\bar{f}_\theta(S)$ nằm trong $\tau_q \pm \delta$. Khoảng 30 lượt mỗi truy vấn, lặp 2–3 vòng.

Vòng 3 — xác nhận bằng model thật. Bắt buộc. Beam search ở §3.6 chạy trên f_θ, nên W là "tối tiểu theo surrogate"; nếu f_θ lệch quanh τ_q thì sai cả bộ mà không gì bắt được.

```
với S* và top-k MWC:  chạy v(S) thật, xác nhận v(S) >= tau_q
                      và với mọi c thuộc S*:  xác nhận v(S* \ {c}) < tau_q

~15 lượt
```

Nguyên tắc giữ nguyên từ §4.10: **không bao giờ để proxy quyết định nhận hay bỏ.**

Ngân sách — đếm bằng token, không phải số lượt. Các lượt gọi *không* bằng giá nhau: prompt chỉ chứa S, nên cost$(S) \approx 500 + 250|S|$ token. Một singleton tốn 750 token, còn $v(C)$ với $|C| = 40$ tốn 10.500 — rẻ hơn hai mươi lần. Mà greedy xây dần từ rỗng nên mọi lượt của nó đều có $|S|$ nhỏ; lượt đắt duy nhất là $v(C)$ và nó chạy một lần. Đếm theo số lượt sẽ ước lượng sai hẳn.

```
n = 40,  C = 6,  m = 12,  r = 3     (ca prior tốt; multi-hop thì m = 30)

v(rỗng) + v(C)                          11.000
singleton x 40                          30.000
greedy   vòng 2:  12 x 1.000            12.000
         vòng 3:  12 x 1.250            15.000
active learning   30 x 1.250            37.500
xác nhận          15 x 1.250            18.750
                                       --------
                                      ~ 124.000 token / truy vấn
```

Ở mức \$3 mỗi triệu token đầu vào thì ≈\$0,37 mỗi truy vấn; 1000 truy vấn huấn luyện ≈\$370, một lần, cache vĩnh viễn. Đối chiếu: quét đầy đủ với $n = 60$ tốn ∼260.000 token — giữ được 40 ứng viên mà rẻ hơn 2,1 lần, và hơn hẳn phương án cắt xuống $n = 20$ về độ phủ.

Vì sao quét hết singleton $(a_0 = n)$ thay vì cũng gom cụm: chúng là lượt rẻ nhất và bắt được ca "một đoạn tự nó đủ" — loại MWC cỡ 1, gọn nhất và đáng tin nhất.

Ba chi tiết cài đặt đáng tiền.

```
cache          khóa = hash( model, template, nội dung chunk đã sắp )
               băm nội dung chứ không phải ID -- chunk lại corpus là ID va nhau

thứ tự chuẩn   xếp chunk theo một thứ tự cố định rồi gom lô theo tiền tố chung:
                   S_1 = {c1, c3, c7}
                   S_2 = {c1, c3, c9}   chung tiền tố "c1 c3" -> chỉ encode đuôi
               Vòng 1 sinh đúng dạng đó: mọi ứng viên trong một vòng là S + {k}
               với cùng S, nên gom cả vòng thành một lô thì tiền tố encode một lần.

lưu kèm        tau_q, vòng, chọn-hay-bị-loại, |S|
               có mấy trường này thì sau còn đánh trọng số lại hướng về biên
```

Chia tập theo cả hai cách, và đọc hai con số khác nhau. Hai cách chia đo hai năng lực khác nhau, mà pha 1 và pha 2 (§3.11) cần đúng hai năng lực đó.

```
giữ lại TẬP CON trong truy vấn đã thấy   = điều kiện pha 1
                                          mô hình có v~_k của truy vấn này; việc nó
                                          "học thuộc" giá trị từng chunk ở đây là
                                          MỤC TIÊU, không phải rò rỉ
giữ lại TRUY VẤN                         = điều kiện pha 2
                                          chưa từng thấy truy vấn, không có v~_k
                                          -> đây mới là con số quyết định §3.11
```

Các bản trước của tài liệu này chỉ nêu cách chia thứ hai và gọi cách thứ nhất là rò rỉ. Điều đó sai: với một truy vấn đã trả vòng 0, nhớ giá trị từng chunk của nó chính là thứ beam search cần.

3.5 Surrogate

Một hàm duy nhất, dùng cho cả hai pha ở §3.11:

$$g_\theta\big(q,\ S,\ \text{đặc trưng mỗi chunk}\big)\ \longrightarrow\ \text{một số thực} = \text{dự đoán } v(S)$$

bất biến hoán vị trên các phần tử của S, vì S là tập.

```
đầu vào   q              văn bản truy vấn, embed
          mỗi k thuộc S  văn bản chunk, embed
                         + s^(q,k)  điểm rerank      LUÔN có, cả hai pha, 0 lượt LLM
                         + v~_k     v({k}) - v(rỗng) chỉ khi đã trả vòng 0
          v(rỗng)        mốc dưới của thang đo       đo trước mọi lần gọi g_theta
```

τ_q không phải đầu vào, và không được phép là đầu vào. g_θ hồi quy lên $v(S)$; việc so với ngưỡng xảy ra sau, bên ngoài mô hình. Ràng buộc này đến từ thứ tự chạy ở §3.11: bước `C2` gọi g_θ để quyết định có mua $v(C)$ hay không, mà $v(C)$ mới là thứ chốt v_{ref} rồi chốt τ_q. Cho τ_q vào đầu vào là tạo ra một vòng tròn thật sự. $v(\emptyset)$ thì ngược lại — nó đo được ở bước `C1`, trước `C2`, nên dùng được.

Dữ liệu huấn luyện. Không thu riêng. Nó *là log của chính các lượt gọi v* ở §3.4 và §3.11 — mỗi lần tính $v(S)$, ghi lại $(q, S, v(S))$.

```
nguồn                              truy vấn   nhãn/truy vấn    tổng
pha 1, bốn vòng (§3.4)              10.000         111        1.110.000
verification đường rẻ (§3.11)      188.000          10        1.880.000
tỉa, |S*| = 6..10 (§3.11)            8.000          13          104.000
cứu -> pipeline đầy đủ               7.400         111          821.000
ca trượt bị bỏ, không cứu           13.600           5           68.000
                                                              -----------
                                                               3.983.000
```

Một mẫu là $(q, S, v(S))$ lưu kèm τ_q, vòng sinh ra, chọn-hay-bị-loại, $|S|$. Cỡ S chạy từ 0 tới 40: $|S| = 0$ là $v(\emptyset)$, $|S| = 1$ là các singleton của vòng 0, $|S| = 12$ là $v(R)$, $|S| = 40$ là $v(C)$. Không cỡ nào đặc biệt — đó là lý do **một** mô hình, chứ không phải một mô hình cho singleton và một mô hình cho phần còn lại.

Nhánh leo thang lớn nhất, và quý nhất. Nó chỉ chiếm 10% truy vấn nhưng đóng gần một nửa số nhãn, vì ba lẽ: log đầy đủ chứ không phải bảy điểm quanh S^*; toàn ca g_θ đoán sai, tức hard negative ở cấp truy vấn; và có $\tilde{v}_k$ thật cho cả 40 chunk vì nó chạy vòng 0.

Che $\tilde{v}_k$ theo từng chunk, trộn ba khuôn. $\tilde{v}_k$ có ở pha 1 mà không có ở pha 2. Train thẳng với nó thì lúc chạy mô hình gặp phân bố khác hẳn; nhưng che tất-cả-hoặc-không-gì cũng sai, vì thực tế có **ba** chế độ:

```
pha 1 + leo thang       cả 40 chunk có v~_k       vòng 0 đầy đủ
verification đường rẻ   CHỈ |S*| chunk có v~_k    3 lượt đơn tử trong 7 lượt kiểm
beam ở bước C5          không chunk nào có        chưa gọi LLM lần nào
```

Trộn ba khuôn theo đúng tỷ lệ chúng xuất hiện. Khuôn thứ hai — biết giá trị của 3 chunk trong S, không biết 37 chunk còn lại — là khuôn mà che tất-cả-hoặc-không-gì không bao giờ dạy được, mà nó lại là khuôn của *mọi* lượt verification ở pha 2.

Mất mát.

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda\, \mathcal{L}_{\text{rank}}$$

$\mathcal{L}_{\text{cls}}$ là BCE trên nhãn $\mathbf{1}[v(S) \geq \tau_q]$ — mục tiêu *chính*, vì §3.6 chỉ cần dấu; $\mathcal{L}_{\text{rank}}$ là hinge theo cặp **trong cùng một truy vấn** để giữ thứ tự thô cho beam.

τ_q xuất hiện ở đây, trong mất mát, chứ không ở đầu vào. Nó định hình mô hình chính xác *ở đâu* — dồn vào biên của từng truy vấn. Để $\mathcal{L}_{\text{cls}}$ là chính chứ không phải MSE trên $v(S)$: hồi quy chính xác ở vùng xa ngưỡng là công sức bỏ phí. Và $\mathcal{L}_{\text{rank}}$ phải so *trong* một truy vấn: so chéo truy vấn thì nó học thứ tự giữa các τ_q khác nhau, mà τ_q khác nhau theo từng truy vấn nên thứ tự đó vô nghĩa.

Nhãn dương chỉ chiếm $\sim$10–15%: greedy đi từ rỗng nên hầu hết tập con nằm dưới ngưỡng. Lệch này là thứ ta muốn — chúng là negative *sát biên* — nhưng phải cân trọng số lớp trong $\mathcal{L}_{\text{cls}}$.

Chỉ số nghiệm thu, và sàn phải vượt. **AUC trong từng truy vấn**, rồi trung bình qua các truy vấn; sàn là **mô hình chỉ nhìn** $|S|$.

AUC gộp mọi truy vấn vào một rổ là cái bẫy. Tập lớn thì v cao — đúng khi xét xuyên truy vấn — nên một mô hình chỉ dùng $|S|$ đã có AUC gộp đẹp. Nhưng nó có năng lực phân biệt trong truy vấn bằng **0**: mọi tập cỡ 3 hòa điểm, beam thành chọn bừa. Không vượt được sàn đó thì g_θ vô dụng dù con số gộp bao nhiêu. Vì lý do tương tự, đừng báo cáo accuracy trần — dồn mẫu ở biên làm nó không so được với accuracy trên mẫu ngẫu nhiên.

Mốc phải vượt trước khi chốt kiến trúc. Phần thực sự phải học là **độ lệch khỏi cộng tính**: hai chunk nói cùng một điều thì không cộng dồn; hai chunk ghép lại mới trả lời được thì cộng hơn tổng.

```
truy vấn 2 hop "ai đạo diễn phim thắng Oscar phim hay nhất 1994?"

                                      v     cộng tính    lệch
v(rỗng)                              0,05
chunk 7  "Forrest Gump thắng 1994"   0,10
chunk 23 "Zemeckis đạo diễn F.Gump"  0,15
S = {7, 23}                          0,95     0,20      +0,75   bắc cầu
S = {7, 8}  (chunk 8 ~ trùng 7)      0,10     0,15      -0,05   trùng lặp
```

$+0{,}75$ và $-0{,}05$ là hai thứ duy nhất mô hình phải đoán, và chúng là *quy luật chung*: hai đoạn gần trùng thì lệch âm, hai đoạn cùng thực thể mà khác quan hệ thì lệch dương. Đây là chỗ gộp nhiều truy vấn có nghĩa.

Bài toán đó nhỏ hơn hẳn việc đoán $v(S)$ từ đầu, nên **một GBM trên đặc trưng tay phải được chạy trước làm mốc**: $|S|$, $\sum_{k\in S} \hat{s}(q,k)$, max $\hat{s}$, số cụm S chạm, tương đồng cặp lớn nhất trong S, tương đồng với q, và $\sum \tilde{v}_k$ khi có. Set Transformer có mặt nạ là kiến trúc đề xuất, lý lẽ là tự chú ý bắt tương tác theo cặp — đúng là hiện tượng cần mô hình hóa. Nhưng đó là lý lẽ, không phải bằng chứng: nó chỉ đáng chọn nếu *vượt mốc GBM trên AUC trong truy vấn*.

Rò rỉ ngữ cảnh pool. Nếu mặt nạ chạy trên cả pool 40 chunk thì mô hình thấy được những chunk *không* nằm trong S, và có thể học dựa vào "cả pool trông thế nào" thay vì vào S. Lúc chấm trong beam thì pool cố định nên đặc trưng đó vô hại; nhưng nó làm con số nghiệm thu đẹp giả, và tệ hơn, nó biến mất đúng lúc pool đổi. Cách chặn: mã hóa S cùng các scalar theo từng chunk, không cho mô hình nhìn phần bù của S trong pool.

3.6 Liệt kê MWC

Bảng phủ (§3.2) làm không gian tìm kiếm co lại rất mạnh:

```
tập KHÔNG phủ hết mảnh  ->  nếu nó vượt tau_q thì chỉ có thể vì mỗi tham số
                        ->  mà ta đã quyết LOẠI ca đó (§3.2)
                        ->  tìm trong vùng đó là tìm thứ ta sẽ bỏ
```

Nên cả dàn 2^n co lại thành **các nghiệm phủ, và mọi tập chứa một nghiệm phủ**. Tìm theo từng tầng, đi lên:

```
tầng 0   các nghiệm phủ tối tiểu (giải chính xác từ bảng phủ)   |T| = số mảnh
tầng 1   nghiệm phủ + 1 đoạn
tầng 2   nghiệm phủ + 2 đoạn
...
mỗi tầng:  chấm bằng g_theta, lấy top-beam, đo v theo thứ tự đó
```

Hai lợi ích, cả hai rơi ra từ cấu trúc: **(1)** mọi ứng viên đã đủ về bằng chứng, không còn ứng viên mồi tham số; **(2)** thứ tự theo cỡ nên tập nhỏ luôn thử trước, tức "tối tiểu trước" là miễn phí theo cấu trúc, không phải một bước tỉa riêng.

Tính tối tiểu gần như không tốn lượt nào. Vì điều kiện nhận có hai mệnh đề (§3.2), một tập con chỉ **đe dọa** tính tối tiểu nếu nó *cũng phủ hết*:

```
T = {3, 12, 19}      3 phủ mảnh 1 · 12 phủ mảnh 2 · 19 là đoạn thêm

tập con thực sự:           7 tập
trong đó CÒN phủ hết:      1 tập  --  {3,12}
     ({3,19} mất mảnh 2 · {12,19} mất mảnh 1 · mọi đơn tử mất một mảnh)

=>  chỉ cần kiểm {3,12},  mà tầng 0 ĐÃ đo nó rồi.
```

Nên nếu tìm theo tầng, các phép đo cần cho tính tối tiểu **chính là các tầng dưới, đã trả tiền**. Không có lượt nào thêm ở cuối — đây là chỗ làm gọn hẳn phần ba ngưỡng $|S^*|$ ở §3.11.

Beam mò vẫn giữ làm dự bị. Bảng phủ có thể thiếu cột (NLI bỏ sót bằng chứng diễn đạt lại), nên nếu mọi tầng đều trượt thì quay về cách cũ, tìm trên toàn dàn:

```
frontier = { rỗng }
cho size = 1, 2, ..., n:
    ứng viên = { S + {i} : S thuộc frontier, i không thuộc S }
    chấm bằng g_theta
    đủ = { S : g_theta(S) >= tau_q }
    nếu đủ khác rỗng:  thu hoạch, và chạy tiếp thêm 1-2 cỡ nữa
    frontier = top-beam theo g_theta

tỉa về tối tiểu:  với mỗi S đủ, bỏ c thuộc S nếu g_theta(S \ {c}) >= tau_q,
                  lặp tới khi không bỏ được
```

Tầng dự bị này chính là nhánh (2) của `C7` (§3.11) — không cần cơ chế mới, chỉ là danh sách ứng viên giờ có hai nguồn.

Ca $v < \tau_q$ dù đã phủ hết — đọc nó cho đúng. Đây là ô dưới-trái của bảng 2×2 (§3.2): bằng chứng *có* trong T mà model vẫn trả lời sai, tức lỗi của **model sinh**, không phải của truy hồi — thêm đoạn là đang "giúp" model, không phải bổ bằng chứng. Nên khi phải đi lên tầng 1, 2 mới qua τ_q, **ghi lại số đoạn thêm**. Con số đó lớn và hay xảy ra thì kết luận không phải "pool thiếu" mà là "model sinh yếu ở bắc cầu" — và tầng B không sửa được điều đó.

Đừng dừng ở cỡ đầu tiên đạt đủ. Dừng ngay thì thiên vị tập nhỏ và có thể bỏ sót một MWC lớn hơn nhưng vững hơn nhiều.

Tỉa phụ thuộc thứ tự. Vì v không đơn điệu, bỏ một phần tử có thể làm v *tăng*, nên phải tỉa nhiều vòng và kết quả phụ thuộc thứ tự bỏ. Chốt một thứ tự và ghi rõ: bỏ theo r_i tăng dần. Đây đúng là căn bệnh mà §4.11 chặng 2 cũng mắc, và cách chữa giống nhau — chốt thứ tự, đừng giả vờ là không có.

Mở rộng trong cụm — trả lại thứ gom cụm đã lấy mất

Lập luận "không cần gom cụm vì dư thừa hiện ra thành bội số MWC" (§3.2) **gãy** khi §3.4 chỉ thăm dò đại diện: nếu c_3 và c_7 cùng cụm mà chỉ c_3 được chấm thì $\{c_{12}, c_7\}$ không bao giờ lộ ra, và tín hiệu PGI mất theo. Chữa bằng một bước chạy **hoàn toàn trên surrogate**, không tốn lượt LLM nào:

```
với mỗi S trong W:
    với mỗi c thuộc S,  với mỗi c' cùng cụm với c,  c' không thuộc S:
        nếu f_theta( S - {c} + {c'} ) >= tau_q:   thêm vào W

xác nhận 2-3 MWC mới bằng model thật
```

Tức **gom cụm để tìm, mở rộng trong cụm để báo cáo**. Hai việc khác nhau, và chỉ việc đầu cần tiết kiệm. Sau bước này thì PGI, overlap và entropy ở §3.7–3.9 quay lại đầy đủ.

3.7 Chỉ số cấu trúc

```
beta_i   Banzhaf có dấu
         beta+   số lần thêm i biến thiếu -> đủ
         beta-   số lần thêm i biến đủ -> thiếu        <- đơn điệu thì bằng 0
         beta_i  =  ( beta+ - beta- ) / 2^(n-1)

PGI_i    số MWC trong W có chứa i
overlap(S)  =  tổng_{S' != S}  |S giao S'| / |S hợp S'|
```

$\beta_i < 0$ **chỉ thẳng ra đoạn gây hại** — đúng vai mà $\phi_c < 0$ đóng trong bản Shapley. Tính không đơn điệu ở đây là thông tin, không phải khuyết tật.

Chuẩn hóa β và PGI riêng rồi mới gộp. $\beta \in [-1, 1]$, còn PGI là một số đếm có thể tới hàng chục; cộng thẳng thì PGI nuốt β một đến hai bậc độ lớn.

$$\sigma_g^{\text{sum}}(S) = \sum_{i \in S} (\tilde{\beta}_i + \widetilde{\text{PGI}}_i), \qquad \sigma_g^{\min}(S) = \min_{i \in S} (\tilde{\beta}_i + \widetilde{\text{PGI}}_i)$$

3.8 Chọn S^*

Ba tiêu chí, chuẩn hóa rồi gộp có trọng số: $\sigma_s(S) = -|S|$ (gọn), $\sigma_g(S)$ (cấu trúc), $\sigma_r(S) = \sum_{i \in S} r_i$ (khớp rerank).

$$S^* = \operatorname*{argmax}_S \; w_s\tilde{\sigma}_s + w_g\tilde{\sigma}_g + w_r\tilde{\sigma}_r$$

Hướng: **vững** thì cực đại σ_g (thưởng bằng chứng dùng được rộng rãi); **đặc thù** thì cực tiểu σ_g. Phải ghi rõ chọn hướng nào — hai hướng cho hai S^* khác nhau. Nên lọc Pareto trước, bỏ các MWC bị trội hoàn toàn. Phá hòa cuối cùng bằng $\sum r_i$, rồi bằng chỉ số từ điển. **Rerank chỉ quyết định ở bước cuối** — cấu trúc lo phần thực chất.

3.9 Credit và độ tin cậy

```
cứng   credit(c_i)  =  1/|S*| nếu c_i thuộc S*,  0 nếu không

mềm    p(S) = softmax( Phi(S)/T )     credit(c_i) = tổng_{S chứa i} p(S)/|S|
       T -> 0   quay về bản cứng
       T -> vô cùng   quay về Deegan-Packel chuẩn (đều trên mọi MWC)
```

Entropy của $p(S)$ là tín hiệu độ tin cậy. Thấp nghĩa là có một người thắng rõ ràng; cao nghĩa là nhiều MWC suýt soát và quy gán đang không chắc. Bản Shapley không có đại lượng tương đương — nó luôn trả về một T duy nhất với vẻ tự tin như nhau.

3.10 Đầu ra

```
T  =  S*                  tập đích, đi sang tầng B
W, p(S)                   họ MWC và phân bố -- để báo cáo
entropy( p )              độ tin cậy
beta_i                    beta < 0 -> ứng viên negative cho V_neg (§6.3)
```

Chỉ T đi qua ranh giới hai tầng; không điểm số nào đi qua — xem §1 của phụ lục *Những chỗ dễ sai*.

Phép kiểm. Năm điều, đo bằng model *thật*:

```
1  v(T) >= tau_q                                 không phải g_theta
2  adq(T) = phủ hết mảnh                         bằng chứng THẬT SỰ nằm trong T (§3.2)
3  v(S') < tau_q  với mọi tập con thực sự S' c T
   mà S' CÒN phủ hết                             T thật sự tối tiểu (§3.6)
4  mọi i có beta_i < 0 đều không nằm trong T      không đưa đoạn gây hại vào đích
5  tổng credit(c_i) = 1                           theo cấu trúc
```

Phép kiểm tối tiểu phải quét hết tập con, không được dùng bỏ-một-phần-tử. Nếu v đơn điệu thì chỉ cần kiểm các tập $S^* \setminus \{c\}$: mọi tập nhỏ hơn đều nằm trong một tập đó, nên đơn điệu cho v(nhỏ hơn) $\le v(S^* \setminus \{c\}) < \tau_q$ miễn phí. Nhưng v **không đơn điệu** (§3.2), nên $v(S^* \setminus \{c\}) < \tau_q$ *không* kéo theo v(tập con nhỏ hơn) $< \tau_q$. Ca cụ thể, với b là chunk gây nhiễu:

```
tau_q = 0,76
v({a})     = 0,85  >= tau_q     ->  {a} MỘT MÌNH đã đủ
v({a,b})   = 0,60  <  tau_q         b làm model lạc
v({a,c})   = 0,71  <  tau_q
v({b,c})   = 0,40  <  tau_q
```

```
v({a,b,c}) = 0,80  >= tau_q

bỏ-một-phần-tử:  cả ba cặp trượt  ->  tuyên "S* tối tiểu"      SAI
sự thật:         {a} thắng        ->  S* thừa cả b và c
```

Chi phí quét hết là $2^{|S^*|} - 2$ lượt (bỏ chính S^*, và $\emptyset$ đã đo sẵn). Sai chỗ này đắt vì T là tập nguồn vàng của tầng B: T thừa thì tầng B học định tuyến tới đoạn không cần, tức nhãn nhiễu theo hướng **thừa**, đúng hướng tệ nhất.

Nhưng không phải quét hết dàn dưới S^* ứng viên — chỉ dưới T cuối cùng. Mục này đòi T *là một* MWC, không đòi nó là MWC duy nhất hay nhỏ nhất. Nên với S^* lớn, cách đúng là **tỉa xuống rồi quét hết ở cuối** (§3.11): nếu $T = \{a, b, c\}$ có $v(T) \geq \tau_q$ và cả 6 tập con thực sự đều dưới ngưỡng, thì T **là** một tập đủ tối tiểu — 247 tập con khác của một S^* cỡ 8 không cần xét, vì ta không khẳng định gì về chúng. Thứ trở nên không đầy đủ là W, và §3.11 đã ghi rằng W ở pha 2 vốn không dùng để kết luận cấu trúc.

Bộ này **yếu hơn** bộ bất biến của bản Shapley — xem §8.

Phân tích độ nhạy. Quét w_s, w_g, w_r mỗi cái $\pm 50\%$ và đếm số lần S^* đổi; quét τ_q quanh $\theta \in [0{,}90;\ 0{,}98]$ và xem W có ổn định không; bootstrap f_θ rồi train lại để lấy phương sai của credit. Nếu S^* lật lung tung khi nhiễu nhỏ thì **báo cáo cả biên Pareto** thay vì một S^* duy nhất.

3.11 Hai pha — mô hình ở pha 1, nhãn ở pha 2

Mọi thứ từ §3.1 tới §3.10 là *một* truy vấn, chạy LLM đầy đủ, $\approx \$0{,}38$. Ở quy mô thật thì không trả nổi: $200.000 \times \$0{,}38 = \$76.000$. Nên tách làm hai pha có mục đích khác nhau.

```
PHA 1   N_1 = 10.000 truy vấn, LLM đầy đủ
        sản phẩm:  g_theta        -- KHÔNG phải nhãn cho tầng B
PHA 2   N_2 = 200.000 truy vấn, đường rẻ
        sản phẩm:  nhãn (q, T) cho tầng B ở quy mô N_2
```

Ở $N_1 = 300$ như các bản trước của tài liệu này thì hai pha trùng nhau nên không cần tách; ở $N_2 = 200.000$ thì pha 1 là **đầu tư vào mô hình** còn pha 2 là **sản lượng**.

Bước truy hồi — chung cho cả hai pha, 0 lượt LLM.

```
3 nguồn -> pool thô K <= 60 -> khử trùng lặp
              |  rerank THEO TỪNG NGUỒN, hạn ngạch riêng (§3.1)
              v
n = 40  =  20 BM25  +  15 GFM  +  5 ngẫu nhiên
              |
              v   gom C cụm,  C <= floor(m/2) (§3.4)
```

Pha 1 — 10.000 truy vấn.

```
A0  chẻ câu hỏi thành mảnh + bảng phủ NLI (§3.2)   400 token
      thiếu mảnh nào không đoạn nào chống đỡ
      -> BỎ truy vấn, cờ "pool thiếu". Đã tiêu 400, không phải 127.500.
A1  vòng 0 -- quét CỐ ĐỊNH, không cần tau_q
      v(rỗng)                                        500 token
      v(R)     R = top-2 mỗi cụm, |R| = 12         3.500 token
      v(C)     cả 40 chunk                        10.500 token
      v({k})   cả 40 chunk đơn                    30.000 token
      -- v_ref = max( v(C), v(R), max_k v({k}) ) ;  tau_q chốt --
```

```
A2  vòng 1 -- greedy biên (§3.4)                  27.000 token
A3  vòng 2 -- active learning quanh tau_q        ~37.000 token
A4  vòng 3 -- xác nhận bằng model thật           ~15.000 token
                                                -------------
                                                 ~127.500 token  ~ $0,38
```

Giữa hai pha.

```
B1  gộp log 10.000 truy vấn       ~1.200.000 mẫu, |S| từ 0 tới 40
B2  train g_theta (§3.5), che v~_k theo ba khuôn
B3  CỔNG p: giữ 500 truy vấn pha 1, chạy TOÀN BỘ đường pha 2 trên chúng,
      so S* thu được với S* thật đã biết
```

B3 dùng nhãn đã có nên **không tốn lượt LLM nào**, và nó là cổng quyết định:

```
p >= 0,90               triển khai 200k
p ~  0,75               tiền cứu ăn gần hết phần tiết kiệm -> tăng N_1 trước
p tụt mạnh theo số hop  g_theta yếu ở ca bắc cầu -> thêm truy vấn multi-hop vào pha 1
p cao mà |S*| phình     phần dư ÂM (trùng lặp) chưa học được
```

Pha 2 — 200.000 truy vấn, đường rẻ.

```
C0  chẻ câu hỏi thành mảnh + bảng phủ NLI (§3.2)          400 token
      thiếu mảnh -> THOÁT, cờ "pool thiếu", bỏ truy vấn
C1  v(rỗng)                                                 500 token
C2  g_theta chấm 40 chunk đơn,  v~_k che                      0 token
C3  v(R)   R = 12 chunk, cách chọn ở dưới                 3.500 token
C4  v(C)   cả 40 chunk                                   10.500 token
      -- v_ref = max( v(R), v(C) ) ;  tau_q chốt --
      v_ref - v(rỗng) < 0,05 -> T = rỗng, cờ "truy hồi không đóng góp" (§3.3)
C5  tìm theo tầng từ nghiệm phủ của C0 (§3.6) -> ứng viên xếp hạng  0 token
      mọi tầng trượt -> beam mò làm dự bị
C6  verification bằng LLM THẬT -- đủ, và tối tiểu          ~7.400 token
C7  ba nhánh: xuất nhãn · thử ứng viên kế · cứu
                                                          -------------
                                                           ~21.900 token  ~ $0,066
```

C2 **phải đứng trước** C5 vì beam cần điểm từng chunk làm prior mở rộng; nó hợp lệ vì g_θ không nhận τ_q (§3.5).

A0/C0 **không tốn thêm tiền — nó tiết kiệm gần một nửa.** Cửa thoát ở C0 tốn 400 token; cửa thoát cũ ở C4 phải trả $v(\emptyset) + v(R) + v(C) = 14.500$ token mới biết. Và theo §3.2, tỷ lệ truy vấn không có tập đủ trong pool là $\sim$45–55% trên các bộ multi-hop:

```
nếu 45% thoát ở C0:
   kỳ vọng  =  0,45 x 400  +  0,55 x 22.300   =   12.400 token  ~ $0,037
   trước đó                                       21.900 token  ~ $0,066
```

Ở pha 1 còn đậm hơn, vì mỗi truy vấn bỏ được là 127.500 token chứ không phải 21.900.

Nhưng truy vấn bị bỏ không phải rác. Chúng là cờ "pool thiếu", nguồn nuôi ô «lỗi đồ thị» §6.1 và vòng lặp ngược §6.6 — và là nguồn *duy nhất* làm ô đó khác rỗng ở quy mô lớn. Ghi lại, đừng xóa.

Cửa thoát phải đặt thận trọng. NLI hay bỏ sót bằng chứng diễn đạt lại hoặc ngầm, nên chỉ thoát khi điểm entail của *mọi* đoạn đối mảnh đó đều rất thấp, không phải chỉ khi nhãn là `neutral`. Và phải lấy mẫu kiểm tay một phần ca bị bỏ — bỏ mù 45% truy vấn dựa trên một model chưa hiệu chuẩn là rủi ro lớn nhất của cả cơ chế này (xem chuẩn ngoài §3.10).

Vì sao mua $v(C)$ chứ không bù bằng một hằng số. Bản trước của mục này bỏ $v(C)$ ở pha 2 và bù $v(R)$ bằng một hằng số hiệu chuẩn đo ở pha 1. Bỏ cách đó, vì ba lẽ:

```
tiền      v(C) chỉ là 10.500 trong 106.000 token mà pha 2 tiết kiệm được -- 10%.
          Phần tiết kiệm thật nằm ở 40 singleton (30.000) và vòng 1-2 (64.000).
nhãn sai  hằng số đặt ở phân vị 90 nghĩa là ~10% truy vấn vẫn bị trần thấp, mà
          trần thấp sinh NHÃN SAI ÂM THẦM -- verification không bắt được, vì nó
          so với chính tau_q đã bị dìm đó.
bộ máy    hằng số đó kéo theo một bước đo riêng, một bước hiệu chuẩn, một cổng
          kích hoạt, và một ký hiệu trùng tên với delta của §3.3.
```

Mua $v(C)$ cho mọi truy vấn: $+\$4.700$ trên 200k, tức $\$0{,}029$ mỗi nhãn, và **pha 1 với pha 2 dùng cùng một định nghĩa v_{ref}**.

Nhưng $v(R)$ vẫn phải giữ. §3.3 đã cảnh báo: $v(C)$ nhồi 40 chunk nên chính nó bị phân tán, một tập con có thể ghi điểm cao hơn cả C, và neo vào $v(C)$ là neo vào *mốc bị dìm*. Pha 1 không gặp vấn đề vì nó lấy max trên hàng chục tập con đã quan sát; pha 2 không quan sát gì, nên cần $v(R)$ — 12 chunk, ít nhiễu hơn hẳn — làm ứng viên thứ hai cho max.

Chọn R: sàn cụm thắng hạn ngạch nguồn. Hai ràng buộc này xung đột thường xuyên, vì cụm gom theo embedding chứ không theo nguồn nên một cụm có thể toàn chunk GFM.

```
6 suất   mỗi cụm 1 chunk, lấy chunk hạng cao nhất trong cụm     SÀN CỤM -- bắt buộc
6 suất   còn lại: trước hết bảo đảm mỗi nguồn có >= 1 trong R
                  rồi lấp phần dư theo hạng
                                                                |R| = 12
```

Sàn cụm thắng vì nhiệm vụ của $v(R)$ là đo **trần**, mà trần phải chạm *mọi* chủ đề. Và 2 chunk mỗi cụm chứ không 1, vì với 1 thì một **cặp bắc cầu nằm trong cùng một cụm** không bao giờ hiện ra — đúng ca 2-hop mà cả thiết kế nhắm tới.

`C6` **— ba ngưỡng theo $|S^*|$.** Phép kiểm tối tiểu phải quét hết tập con (§3.10), mà chi phí đó nổ theo $2^{|S^*|}$. Lối ra không phải bỏ phép kiểm, mà là **tỉa xuống trước rồi mới quét**.

```
S <- S* ứng viên
lặp khi |S| > 5:
    xếp c thuộc S theo g_theta(q, S bỏ c) giảm dần          0 lượt LLM
    thử c đầu bảng:  đo v(S bỏ c)
        >= tau_q  ->  S <- S bỏ c,  lặp
        <  tau_q  ->  thử c kế tiếp, tối đa 3 lần
    3 lần đều trượt  ->  KẸT
|S| <= 5  ->  quét hết dàn dưới S  ->  T, đã CHỨNG MINH
```

Tỉa rồi quét vẫn là chứng minh đầy đủ. §3.10 chỉ đòi T *là một* MWC, không đòi nó là MWC duy nhất hay nhỏ nhất. Nếu $T = \{a, b, c\}$ có $v(T) \geq \tau_q$ và cả 6 tập con thực sự đều dưới ngưỡng, thì T **là** một tập đủ tối tiểu — 247 tập con khác của một S^* cỡ 8 không cần xét, vì ta không khẳng định gì về chúng. Cái trở nên không đầy đủ là W, mà W ở pha 2 vốn đã không dùng để kết luận cấu trúc.

Chi phí quét hết nổ theo 2^s, còn tỉa chỉ tăng **tuyến tính** theo s:

```
|S*|   quét hết ngay    tỉa rồi quét ở cuối    cứu
 3        $0,020               --             $0,34
 5        $0,107            $0,107            $0,34
 6        $0,239            $0,043            $0,34
 8        $1,14             $0,071            $0,34
10        $5,37             $0,098            $0,34
15        $172              $0,218            $0,34
20        $5.500            $0,326            $0,34
```

Nên ba ngưỡng:

```
|S*| <= 5         quét hết ngay                      $0,02 - $0,11
6 <= |S*| <= 10   tỉa xuống <= 5 rồi quét hết        $0,04 - $0,10
|S*| >= 11        cứu                                $0,34
tỉa KẸT ở >= 6    cứu                                $0,34
```

Tỉa còn rẻ hơn cứu tới $s \approx 20$, nên ranh giới $10/11$ *không* đặt theo tiền: ở cỡ đó ta không còn tin điểm xuất phát đủ để tỉa 15 lượt theo chỉ dẫn của chính mô hình đang sai. Đây là một núm mà B3 đo được rồi chỉnh — đo tỷ lệ tỉa-kẹt theo $|S^*|$.

Tỉa phụ thuộc thứ tự bỏ, vì v không đơn điệu — đúng căn bệnh của §3.6, cách chữa giống nhau: chốt một thứ tự (ở đây là g_θ giảm dần) và ghi rõ.

Tỉa chỉ đi xuống được. Nếu S^* vừa phình vừa *thiếu* một chunk cần thiết thì mọi tập con đều trượt và tỉa đứng tại chỗ. Kẹt ở $|S| \geq 6$ nghĩa là g_θ sai cả hai chiều — đó là lúc cứu, và vì vậy tỉa **không thay thế** cứu, nó **lọc** xem truy vấn nào thật sự cần cứu.

C7 **— ba nhánh.**

```
(1) v(S*) >= tau_q, và quét ra được một tập tối tiểu
       ->  T, xuất sang tầng B

(2) v(S*) < tau_q                      S* KHÔNG đủ -- g_theta đoán cao quá
       ->  thử ứng viên kế của beam, tối đa 3 lần
           mỗi lần thử chỉ 1 lượt (hết đủ thì dừng, không kiểm tối tiểu)
       ->  qua -> nhánh (1);   3 lần đều trượt -> nhánh (3)

(3) |S*| >= 11,  tỉa kẹt ở >= 6,  hoặc 3 lần thử đều trượt
       ->  hàng cứu có phân tầng
```

Ca "đủ mà không tối tiểu" không vào hàng cứu. Khi quét hết, C6 đo *trọn vẹn* dàn tập con dưới S^* — với $|S^*| = 3$ là cả 8 tập con, $|S^*| = 4$ là cả 16 — nên các phần tử tối tiểu trong đó đọc ra ngay, 0 lượt thêm:

```
v({a,b,c}) = 0,80 >= tau_q      S* đủ
v({a,b})   = 0,78 >= tau_q      tập con cũng đủ  ->  S* KHÔNG tối tiểu
v({a}) = 0,30   v({b}) = 0,22   cả hai đơn tử dưới ngưỡng
->  T = {a,b}
```

Đây là ca xảy ra thường xuyên, vì lỗi hay gặp nhất của g_θ là đề xuất tập **phình**.

Nhánh (2) rẻ hơn tưởng. Hết đủ thì dừng ngay, nên một ca trượt chỉ tốn 1 lượt (1.250 token) chứ không phải 7. Và một lượt cứu tốn 113.000 token — **mua được 90 lần thử bằng giá một lần cứu**. Dừng ở 3 vì ứng viên sau cùng một mô hình nên cùng thiên lệch. Với tỷ lệ qua có điều kiện $\sim$30% mỗi lần thử, 3 lần vớt $1-0{,}7^3 = 66\%$ số ca, nên p hiệu dụng $0{,}75 \to 0{,}75+0{,}25\times0{,}66 = 0{,}91$.

Đây *không* phải đi tìm ngưỡng dễ: mỗi ca qua đều là $v(S') \geq \tau_q$ **đo thật**, rồi tối tiểu vẫn bị quét hết. Nhưng phải **ghi lại hạng beam được nhận**, vì nhận ứng viên #3 đẩy phân bố $v(S^*) - \tau_q$ xuống sát ngưỡng hơn, và tầng B cần biết để đánh trọng số.

Cứu là gì. **Cứu = chạy** `A1–A4` **lên truy vấn đó**, tức pipeline đầy đủ với LLM thật, thay vì tin vào đề xuất của g_θ. Nó không bắt đầu từ 0:

```
đã có trong cache    v(rỗng) 500 · v(R) 3.500 · v(C) 10.500          14.500
                     + v(S*) và dàn tập con đã đo lúc C6
còn phải chạy        40 singleton                                    30.000
                     vòng 1 greedy biên                              27.000
                     vòng 2 active learning                          37.000
                     vòng 3 xác nhận                                  6.500
                                                                   ---------
                     cứu, tính biên                                 113.000  ~ $0,34
```

τ_q có thể TĂNG khi cứu, và đó là đúng. Pha 2 chỉ quan sát $v(\emptyset)$, $v(R)$, $v(C)$; cứu quan sát thêm 40 singleton và hàng chục tập con greedy, nên v_{ref} ở cứu lấy max trên cả $\max_k v(\{k\})$ và mọi $v(S)$ quan sát trong greedy. v_{ref} chỉ có thể lên, nên τ_q lên theo: một truy vấn trượt ngưỡng pha 2 có thể gặp ngưỡng *cao hơn* lúc được cứu. Nghe nghịch lý nhưng đúng — pha 2 đang đo trần thiếu, và cứu sửa chỗ đó.

Cứu mà vẫn trượt là ca có giá trị nhất. Greedy chạy hết, active learning xong, không tập nào vượt τ_q: báo $T = \emptyset$ kèm cờ *"pool không chứa tập đủ"*. Đây không phải thất bại của g_θ mà là bằng chứng **pool thiếu** — đúng thứ thiết kế đa nguồn sinh ra để phát hiện. Nó nuôi ô «lỗi đồ thị» của bảng chẩn đoán §6.1 và vòng lặp ngược §6.6. Nếu ô đó luôn rỗng thì hoặc đồ thị hoàn hảo, hoặc ta đang không đo được nó; ca cứu-mà-trượt là nguồn duy nhất ở pha 2 làm nó khác rỗng.

Cứu phải lấy mẫu, và phân tầng theo cái gì. Cứu hết thì $\sim$17.000 ca trượt $\times$ \$0,34 = \$5.780 để lấy 17.000 nhãn — \$0,34 mỗi nhãn, đắt gấp 5 đường rẻ. Nhưng tính theo **mẫu huấn luyện** thì ngược lại: 111 mẫu/\$0,34 = \$0,0031 mỗi mẫu, so với 10 mẫu/\$0,066 = \$0,0066. Nên cứu không phải để lấy *cái nhãn đó*.

```
ĐO     "bao nhiêu % ca trượt là g_theta sai, bao nhiêu là pool thiếu?"
       -> cần mẫu KHÔNG LỆCH: ngẫu nhiên trong từng tầng, kèm trọng số N_s/n_s
TRAIN  nuôi g_theta đúng chỗ nó yếu
       -> muốn càng nhiều càng tốt, và lệch về tầng khó nhất
```

Cách chia: một hạn mức nhỏ cố định cho mục đích **đo** (ví dụ 50 ca mỗi tầng), rồi phần còn lại dồn hết cho tầng khó để **train**.

```
tầng                          ý nghĩa                          ưu tiên cứu
trượt, hỏng ở ứng viên #1     g_theta lệch mạnh                 cao
trượt, hỏng cả 3 ứng viên     g_theta sai hệ thống              CAO NHẤT
tỉa kẹt ở |S| >= 6            sai cả hai chiều                  CAO NHẤT
|S*| >= 11                    phình nặng                        cao
nhiều hop                     ca bắc cầu -- đúng thứ ta cần     CAO NHẤT
v(S*) sát ngay dưới tau_q     có thể chỉ là trần đo thiếu       thấp
phiên bản g_theta^(b)         để tách xem lô nào yếu            (chỉ để đo)
```

Ca bị bỏ vẫn không vô giá trị. Mỗi ca trượt không được cứu vẫn xuất $v(\emptyset)$, $v(R)$, $v(C)$, $v(S^*)$ và dàn tập con đã đo — $4+$ nhãn thật — cộng với sự kiện *"g_θ đề xuất S^* mà LLM bác"*, là hard negative ở cấp truy vấn. Ghi vào log train; chỉ **không** xuất T.

Dấu xuất xứ trên mỗi nhãn.

```
T = S*
+ phiên bản g_theta    g_theta^(1) ... g_theta^(10)   lô nào gán
+ đường đi            qua trực tiếp · giải trong dàn · tỉa · thử lại #k · được cứu
+ trọng số importance  N_s / n_s của tầng lấy mẫu
+ v(S*) - tau_q        khoảng vượt ngưỡng
```

Có bốn trường này thì tầng B lọc được xuống tập tin cậy nhất ("chỉ pha 1 + ca được cứu") khi cần, hoặc đánh trọng số lại. Không có thì nó train trên một tập chất lượng không đều mà không có cách nào biết.

Chạy theo lô, không một lượt.

```
lô b    chạy 20.000 truy vấn bằng g_theta^(b)
        |- qua    ->  xuất T sang tầng B
        +- trượt  ->  hàng cứu
        lấy mẫu hàng cứu theo tầng  ->  chạy pipeline đầy đủ        <- ở ĐÂY
        gộp mọi nhãn mới  ->  train  g_theta^(b+1)
lô b+1  ...
```

Cứu phải xử lý **ở ranh giới lô**, cùng lúc với train lại, vì nhãn cứu là đầu vào giá trị nhất cho $g_\theta^{(b+1)}$. Dồn hết sang cuối chương trình thì chúng không dạy được gì cho các lô sau và mất toàn bộ bánh đà. Nhãn mà một lô sinh ra nằm **đúng chỗ** g_θ **yếu**: chúng là các tập con do chính nó đề xuất rồi bị LLM bác, tức đúng vùng phân bố mà beam tra tới mà greedy chưa từng đi qua.

```
lô    truy vấn   trượt sau thử lại   tỷ lệ cứu   số ca cứu
 1     20.000         ~9%   1.800        40%        720
 5     20.000         ~7%   1.400        25%        350
10     20.000         ~5%   1.000        10%        100
                                                   -----
                                                  ~3.400  (+ ca |S*|>=11 và tỉa kẹt)
```

Giảm dần vì p lên thì mỗi ca khó bớt giá trị biên. Nhưng **sàn 5% phải giữ**, không bao giờ xuống 0 — nếu không thì ô «lỗi đồ thị» mất nguồn và ta không còn đo được tỷ lệ trượt thật. Và lý do chính của việc chia lô *không phải* tiền: mỗi ranh giới lô là một **cửa thoát**.

```
sau lô 1   p = 0,70 như dự  ->  chạy tiếp
           p = 0,45         ->  DỪNG. mới tốn ~$1.900, chưa tốn $19.000
           |S*| phình đều   ->  dừng, sửa phần dư âm trước khi chạy tiếp
```

Một chỗ dễ dùng sai trọng số importance.

```
ĐO     p, tỷ lệ trượt, mọi đại lượng quần thể  ->  PHẢI dùng trọng số
                                                   (chỉ cứu 10-40%, mẫu lệch)
TRAIN  g_theta                                 ->  ĐỪNG hạ trọng số ca leo thang
                                                   chúng là hard example, muốn chúng nặng
```

Hai mục đích ngược chiều nhau trên cùng một tập dữ liệu. Dùng chung một bộ trọng số cho cả hai là lỗi âm thầm: mô hình vẫn train, chỉ là nó học nhẹ đi đúng ở chỗ nó yếu.

Chất lượng W và β ở pha 2 thấp hơn pha 1. W, $p(S)$, entropy, β_i ở §3.7 đều đến từ beam trên g_θ, và ở pha 2 verification *chỉ kiểm* T, không kiểm cả họ W. Nên $\beta_i < 0 \to V_{\text{neg}}$ (§6.3) ở pha 2 là bằng chứng yếu hơn hẳn; nếu V_{neg} quan trọng thì lấy nó từ 10.000 truy vấn pha 1.

Thêm nữa, T ở pha 2 là một MWC *trong* S^*, không phải MWC toàn cục của C: có thể còn tập thắng nhỏ hơn nằm ngoài S^* mà ta không bao giờ thấy, vì C5 chỉ beam chứ không liệt kê hết. Pha 1 cũng vậy nhưng nhẹ hơn, vì nó có greedy thật và active learning quét quanh biên.

Sổ chi phí.

```
pha 1    18.200 nạp vào, 10.000 chạy đủ     =   $3.850
pha 2   200.000 nạp vào, 110.000 chạy đủ    =   $7.500
cứu     ~3.000 ca x $0,34                   =   $1.000
hiệu chuẩn 500 HotpotQA (§3.10)             =     $190
                                             ---------
                                               $12.540     ~104k nhãn ĐÁNG TIN

chạy pipeline đầy đủ cho cả 200k               $76.000
```

Khoảng **3,9 lần rẻ hơn**. Phần tiết kiệm đến từ việc bỏ vòng 0 ở pha 2 — 40 lượt singleton $(30.000$ token$)$ thay bằng g_θ, cộng vòng 1–2 (64.000) — *không* từ việc cắt verification hay bỏ $v(C)$.

```
truy vấn THOÁT ở C0  (~45%)
C0  chẻ + bảng phủ            400 token  100%

truy vấn CHẠY ĐỦ  (~55%)
C0  chẻ + bảng phủ            400 token    2%
C1  v(rỗng)                   500 token    2%
C3  v(R)                    3.500 token   16%
C4  v(C)                   10.500 token   47%
C6  verification, kỳ vọng   7.400 token   33%
                           -------------
                           22.300 token

kỳ vọng mỗi truy vấn nạp vào: 0,45x400 + 0,55x22.300 = 12.400 token ~ $0,037
```

Cắt C6 thì T không tối tiểu vẫn được xuất nhãn, và tầng B học định tuyến tới đoạn không cần — nhãn nhiễu theo hướng **thừa**, đúng hướng tệ nhất (§3.10). Còn C0 thì ngược lại: nó là bước *duy nhất* trong cả pha 2 mà bỏ đi sẽ làm chi phí *tăng*.

Bảng này chưa được kiểm chứng — nói thẳng.

```
p = 0,75              GIẢ ĐỊNH, chưa đo. toàn bộ cột chi phí treo vào nó.
30% mỗi lần thử lại   GIẢ ĐỊNH. nếu thấp hơn thì p hiệu dụng 0,91 không đạt.
~4% có |S*| >= 6      minh họa, theo lý thuyết 2-3 hop. chưa đo.
|S*| = 3 la cot chinh chưa đo trên dữ liệu thật.
```

Cổng B3 tồn tại chính vì bốn dòng trên. Chạy nó **trước** khi cam kết N_2; nó miễn phí.

4 Tầng B — Tìm đường trên đồ thị

Đầu vào của tầng này là tập đích T vừa ra ở §3.8. Mọi thứ từ đây chạy trên GFM 34M, không đụng tới LLM nữa.

Bài toán. T nói *đoạn nào* cần; nó không nói *đi lối nào* tới đó. Biết model xếp sai D thì phải dạy lại — nhưng dạy nó cái gì?

Mục tiêu. Với mỗi $D \in T$, tìm một tập đường từ seed tới D: nhỏ, đọc hiểu được, và kiểm chứng được — bật riêng nó lên thì model vẫn tìm ra D.

Trở ngại. G-reasoner cộng tổng thông điệp, không có trọng số cạnh nào để quy gán. Và tín hiệu đi qua L tầng nên credit assignment loãng dần.

Công cụ: Integrated Gradients

IG trả lời một câu hỏi: *một đầu ra vô hướng F chịu ảnh hưởng bao nhiêu từ mỗi đầu vào?* Lấy gradient tại điểm đang xét là cách hiển nhiên, nhưng nó hỏng khi hàm bão hòa. IG sửa bằng cách đi từ một mốc "không có gì" tới điểm thật, cộng dồn gradient dọc đường:

$$\mathrm{IG}_i \;=\; \underbrace{(x_i - b_i)}_{\text{đi xa mốc bao nhiêu}} \;\cdot\; \underbrace{\int_0^1 \frac{\partial F}{\partial x_i}\Big|_{x=b+\alpha(x-b)} d\alpha}_{\text{gradient dọc đường, không chỉ ở đích}}$$

Ví dụ nhỏ cho thấy vì sao cần tích phân. Lấy $F(a,b) = \min(a+b, 1)$, mốc $(0,0)$, điểm thật $(0{,}8;\ 0{,}6)$:

```
gradient tại điểm thật:   a+b = 1.4 > 1, F đã bão hòa  ->  dF/da = dF/db = 0
                          gradient kết luận "cả hai đều vô dụng" -- sai hẳn

IG đi dọc đường (0.8a, 0.6a):  alpha < 0.71 thì chưa bão hòa, gradient = 1
                               alpha > 0.71 thì bão hòa,      gradient = 0

    IG_a = 0.8 x 0.71 = 0.57
    IG_b = 0.6 x 0.71 = 0.43
    tổng = 1.00 = F(0.8,0.6) - F(0,0)
```

Dòng cuối là tính chất quan trọng nhất, gọi là *completeness*: các phần cộng lại đúng bằng cái tổng. Nhờ nó mà mọi con số IG đọc được như "chiếm bao nhiêu phần trăm", và mọi phép kiểm trong tài liệu này đều dựa vào nó.

> **Hai nghĩa của chữ "đường", đừng lẫn.** *Đường tích phân* là đoạn thẳng trong **không gian đầu vào**, đi từ mốc tới điểm thật — một thiết bị toán học bên trong công thức IG. *Đường trên đồ thị* là chuỗi cạnh seed $\to \cdots \to D$, thứ ta đang đi tìm. Tài liệu nói "đường" mà không nói rõ thì luôn là nghĩa thứ hai.

Áp vào bài toán này — nhìn tổng thể

G-reasoner không có trọng số cạnh, nên trước hết phải *tạo ra đầu vào để IG chấm*: gắn một gate vô hướng lên mỗi thông điệp. Tắt hết gate là mốc, bật như model thật là điểm thật. Rồi ba bước:

```
1  đo    IG chia cái bánh "nhờ đồ thị mà p(D) cao lên" xuống từng (cạnh, tầng)
         ->  một bảng số

2  nối   bảng số chưa phải lời giải thích. Ghép các ô điểm cao thành chuỗi liền
         mạch theo thứ tự tầng -- cạnh nào ở tầng 1, cạnh nào ở tầng 2, ...
         ->  một đường đi

3  thử   IG là đạo hàm: nó nói "vặn mạnh thì p(D) tăng", không nói "chừng này đủ".
         Bật riêng đường đó rồi chạy model, xem có còn tìm ra D không.
         ->  nhận hoặc bỏ
```

Ba bước đó chính là ba phần của mục này:

```
T  (tầng A: đoạn nào cần)
 |
 |   A. Đo       §4.1-4.5      hỏi gì -> gắn gate -> chạy IG -> tự kiểm bảng số
 v
bảng IG:  mỗi (cạnh, tầng) một con số
 |
 |   B. Nối      §4.6-4.9      chuỗi là gì -> vẽ lại đồ thị -> quét -> vì sao cộng
 v
đường đi tới một đích
 |
 |   C. Thử      §4.10-4.12    nhận hay bỏ -> một đích nhiều seed -> nhiều đích
 v
Pi, depth(D), w^        nhãn train cho tầng B
```

Viết theo G-reasoner [10]: QuadGraph bốn tầng node (attribute / entity / document / community), GNN phụ thuộc truy vấn, 34M tham số. Đầu ra của nó là top-k node mỗi loại, rồi các node tài liệu được nhét vào prompt cho LLM sinh câu trả lời.

Ký hiệu mục này giới thiệu.

```
V, E(script), R  node; cạnh; quan hệ.      Một cạnh: e = (v', r, v)
L, V_q           số tầng GNN (3-6); tập node seed trích từ truy vấn
v_D              node đồ thị ứng với tài liệu D
h_v^(l), h_r^(l) embedding node v sau l tầng; embedding quan hệ r
h_q              embedding truy vấn
m_e^(l)          thông điệp mà cạnh e mang ở tầng l
g_e^(l), ghat    gate; gate đã học, phụ thuộc truy vấn (chế độ 2)
g*               điểm cuối khi đo: luôn = 1 để đào nhãn; = ghat chỉ khi làm báo cáo
eps_g, E_act     ngưỡng lọc gate (0.05); tập (cạnh, tầng) còn lại sau lọc
p(v), p^(l)(v)   điểm node ở tầng cuối; điểm node ở tầng l
F_B(g)           điểm mục tiêu tầng B = log p(v_D)
M, alpha_m       số bước nội suy (20-50); tham số nội suy, alpha_m = m/M
IG_e^(l), what   quy gán cho cạnh; IG chuẩn hóa [0,1] làm nhãn L_edge
h = |pi|         số hop (số cạnh) của một đường
Vhat, Ehat       node / cạnh của đồ thị trải phẳng theo tầng
best[(v,l)]      giá trị QHĐ tốt nhất tới ô (v,l)
back[(v,l)]      con trỏ truy ngược
pi, pi*_D        một đường; đường tốt nhất tới đích D
Pi_D, Pi         cây đường của đích D; toàn bộ cây      <- đầu ra tầng B
eps_tilde, eps   phần cái bánh tối thiểu một đường phải mang (0.10); eps = eps_tilde*cake
n_max            trần số đường, suy ra: ceil(theta_s/eps_tilde) = 10
sigma(D)         số seed với tới được D (§4.8)
E_reach          cạnh có thể nằm trên một đường từ seed
suff_max         trần của suff do credit mồ côi (§4.10)
depth(D)         tầng đầu tiên mà D tới được (§5.2)
Ecov             tập cạnh đã nằm trong báo cáo (§5.3)
suff, nec        đo đủ; đo thiết          theta_s   ngưỡng suff (0.95)
```

A. Đo cạnh

Biến câu hỏi thành một bảng số, và tự kiểm bảng đó.

4.1 Điểm mục tiêu

$$p(v) = \text{Predictor}_{\text{type}(v)}\big(h_v^{(L)}, \text{emb}(v), \text{emb}(q)\big), \qquad F_B(g) = \log p(v_D) \quad \text{với mỗi } D \in T$$

F_B là hàm của vector gate g, không phải của embedding. Đó là khác biệt cốt lõi so với tầng A.

4.2 Gate trên cạnh

G-reasoner tính thông điệp bằng DistMult rồi cộng tổng: không attention, không hệ số riêng cho từng cạnh. IG cần một biến liên tục vô hướng để lấy đạo hàm, nên ta chèn một hệ số nhân vào mỗi thông điệp.

Thông điệp là vector d chiều, gate là một vô hướng trong $[0, 1]$. Một gate điều khiển cả vector, nhờ đó mỗi cạnh chỉ có một đại lượng cần quy gán thay vì d đại lượng. Gate đánh chỉ số theo cặp (cạnh, tầng), tổng cộng $|\mathcal{E}| \times L$ gate; lý do tách theo tầng ở §4.3.

$$\underbrace{m_e^{(l)}}_{\text{vector}} = \underbrace{g_e^{(l)}}_{\text{vô hướng}} \cdot \text{Msg}\big(\underbrace{h_v^{(l-1)}}_{\text{đích}}, \underbrace{h_r^{(l)}}_{\text{quan hệ}}, \underbrace{h_{v'}^{(l-1)}}_{\text{nguồn}}\big), \qquad h_v^{(l)} = \text{Upd}\left(h_v^{(l-1)}, \textstyle\sum_{e \text{ vào } v} m_e^{(l)}\right)$$

Ba dạng của gate:

```
g         gate nói chung; ở chế độ 1 luôn bằng 1
ghat      gate đã học ở chế độ 2
g(alpha)  gate vặn theo alpha khi chạy IG (§4.4):  g(alpha) = alpha * ghat
```

Chế độ 1 — gate là công cụ phân tích. Đặt $g_e^{(l)} = 1$ cho mọi cạnh, mọi tầng; model chạy y hệt bản gốc, gate chỉ tồn tại để IG có chỗ bám. Dùng ở vòng khởi động (§6.5).

Chế độ 2 — gate là bộ phận của model, phụ thuộc truy vấn.

$$\hat{g}_e^{(l)} = \sigma\Big(\text{MLP}\big(h_v^{(l-1)},\, h_r^{(l)},\, h_{v'}^{(l-1)},\, h_q\big)\Big) \in (0, 1)$$

Ba đầu vào đầu mô tả cạnh; đầu vào thứ tư là embedding truy vấn, cho phép model định tuyến khác nhau theo từng truy vấn. Bản gốc không có cơ chế này: truy vấn chỉ tác động lúc khởi tạo, sau đó L tầng lan truyền chạy như nhau bất kể hỏi gì.

Gate vẫn inductive — cả bốn đầu vào của MLP suy từ văn bản hoặc quan hệ, không có ID node.

4.3 Gate phải riêng từng tầng

Một cạnh vật lý được dùng L lần, mỗi tầng một lần — và các lần đó không tương đương:

```
X --sinh_ở--> C --thuộc--> D          seed = {X}

tầng 1   X->C nổ  ->  C nhận tín hiệu truy vấn         hữu ích
         C->D nổ  ->  C còn trống, D nhận đồ vô nghĩa  vô dụng

tầng 2   C->D nổ  ->  C đã có tín hiệu, D nhận được    hữu ích
         X->C nổ  ->  C đã có rồi, thừa                gần vô dụng
```

Gate chung sẽ trộn hai vai trò đó lại và mất thông tin thứ tự — tức mất đúng cái quan trọng nhất của multi-hop. Nên số gate là $|\mathcal{E}| \times L$.

4.4 IG trên gate

Đường tích phân vặn mọi gate theo một tham số chung $g(\alpha) = \alpha \cdot g^*$, với $\alpha = 0$ là trạng thái không thông điệp nào chảy (mọi node giữ $h^{(0)}$) và $\alpha = 1$ là đồ thị chạy bình thường. Quy gán cho mỗi cặp (cạnh, tầng):

$$\mathrm{IG}_e^{(l)} = \underbrace{\left(g_e^{*(l)} - 0\right)}_{\text{đầu vào} - \text{baseline}} \times \underbrace{\frac{1}{M}\sum_{m=1}^{M} \frac{\partial F_B\left(\alpha_m g^*\right)}{\partial g_e^{(l)}}}_{\text{gradient trung bình dọc đường}}$$

Baseline bằng 0 nên thừa số đầu rút gọn thành $g_e^{*(l)}$. Đây là dạng chuẩn của IG, chỉ khác biến được móc vào: tầng A móc vào embedding token, tầng B móc vào gate cạnh.

Đo thì luôn đặt $g^* = 1$, kể cả sau khi model đã có gate học được. Thừa số đầu thành 1 và rụng hẳn, còn lại đúng gradient trung bình. Lý do ở ngay dưới.

> KIỂM TRA 4 $\quad \sum_{e,l} \mathrm{IG}_e^{(l)} = F_B(g^*) - F_B(0)$

Cái bánh của tầng B lấy ở đâu ra. Tổng $\sum$IG không phải cộng từ bảng IG — nó là hiệu của **hai lượt chạy**, biết được trước cả khi chạy IG:

```
lượt 1   g = 0     p(D) = 0.0183     F_B(0)    = -4.00
lượt 2   g = g*    p(D) = 0.0412     F_B(g*)   = -3.19
                                     --------------------
                              cái bánh = 0.81 nat
```

tức lan truyền trên đồ thị nhân xác suất của D lên $e^{0,81} = 2{,}25$ lần.

$F_B(0)$ không bằng 0: tắt hết gate thì node mất tin từ hàng xóm, nhưng predictor vẫn nhận được emb(v) và emb(q) nên vẫn so khớp văn bản được. Vậy $F_B(0)$ là điểm đạt được *chỉ bằng so khớp văn bản, không dùng cấu trúc đồ thị*, còn cái bánh là phần mà *cấu trúc đồ thị* đóng góp thêm — cùng khuôn với tầng A, nơi $F(E')$ là "chỉ tri thức tham số". IG chỉ làm một việc: chia cái bánh đó xuống từng cặp (cạnh, tầng).

Vì sao đo thì luôn $g^* = 1$

Từ mốc 2 model có gate học được, và sẽ rất cám dỗ khi đặt $g^* = \hat{g}$ — đo đúng chỗ model đang vận hành. Nhưng làm vậy thì hỏng việc, vì thừa số đầu chính là $\hat{g}$:

canh	ĝ	gradient trung binh	IG = ĝ x gradient
e1	0.90	0.30	0.27
e2	0.02	0.85	0.017

e_2 nhạy gấp gần ba lần e_1 — mở nó ra thì $p(D)$ tăng mạnh — nhưng vì model đang đóng nó, IG của nó nhỏ hơn 16 lần và nó rơi khỏi bảng.

Mà công việc đào nhãn chính là **tìm cạnh model lẽ ra nên mở**. Cạnh đó theo định nghĩa có $\hat{g} \approx 0$: nếu model đã mở rồi thì nó đã không xếp sai. Đặt $g^* = \hat{g}$ tức là xóa sạch đúng những thứ cần tìm, và $\mathcal{L}_{\text{edge}}$ khi đó chỉ dạy gate củng cố chính nó.

> $\hat{g}$ là thứ **model làm**, không phải thứ ta dùng để **đo**.

Nên ở mọi mốc, mọi lượt đào nhãn: $g^* = 1$. Chế độ 2 xuất hiện trong model, không xuất hiện trong cách đo.

Một ngoại lệ: báo cáo trung thực. Khi cần giải thích cho người dùng "hệ thống đã lấy D qua lối nào", báo cáo không được kể tên cạnh model chưa từng dùng — lúc đó mới đặt $g^* = \hat{g}$. Đây là tính năng phụ, không nằm trong vòng train. Lượt báo cáo còn được một prior miễn phí để lọc trước:

```
E_act = { (e,l) : g^_e^(l) > eps_g }      eps_g ~ 0.05

chi chay QHĐ tren E_act  ->  thuong con vai phan tram so canh
```

Lượt đào nhãn không dùng được prior này — cạnh cần tìm nằm đúng ở $\hat{g} \approx 0$, tức ngoài E_{act} theo định nghĩa. Nó phải quét tất, đúng bằng chi phí chế độ 1 ở §7.

> Chạy $g = 1$ trên model đã có gate nghĩa là đánh giá nó ở cấu hình nó không vận hành thật. Về ngữ nghĩa thì đúng — đó là mốc "mọi thông điệp chảy đủ", cũng chính là hành vi của model trước khi thêm gate ở mốc 2.

4.5 Tính chất chia đều — kiểm tra nội bộ miễn phí

Một đường chỉ truyền được tin khi mọi gate trên nó đều mở: đóng một gate ở giữa là cả đường vô dụng, dù các gate còn lại mở hết. Nên đóng góp của đường vào F_B là **tích** của các gate, không phải tổng:

$$\text{số hạng của đường} \;=\; c \prod_{i=1}^{h} \frac{g_i}{g_i^*}$$

Chia cho g_i^* chỉ để chuẩn hóa: tại $g = g^*$ mọi thừa số bằng 1, số hạng bằng đúng c. Với lượt đào nhãn thì $g^* = 1$ nên phép chia này biến mất luôn. IG vặn tất cả gate bằng một tham số chung $g = \alpha g^*$, nên mỗi thừa số thành α và có h thừa số — số hạng thành $c\,\alpha^h$. Đường càng dài càng tắt nhanh khi α giảm. Đạo hàm theo g_i rồi tích phân:

$$\text{IG}_i = g_i^* \int_0^1 \frac{c\,\alpha^{h-1}}{g_i^*}\, d\alpha = c \left[\frac{\alpha^h}{h}\right]_0^1 = \frac{c}{h}$$

g_i^* triệt tiêu, nên kết quả không phụ thuộc gate đó mở to hay nhỏ — và cũng không phụ thuộc ta chọn điểm cuối nào. Chỉ c đổi nghĩa: với $g^* = 1$ nó là giá trị đường khi mọi gate mở hết, với $g^* = \hat{g}$ là giá trị tại định tuyến thật của model.

KIỂM TRA 5 đường h hop, tổng giá trị c $\implies$ mỗi hop nhận c/h

Tích phân chỉ xác nhận một điều vốn đã bị ép buộc: mọi hop trên đường đều thiết yếu như nhau — bỏ bất kỳ cái nào thì đường chết — nên chúng đối xứng hoàn toàn; completeness thì ép tổng phải bằng c. Đối xứng cộng với tính đầy đủ đã khóa chặt đáp án. (Lưu ý h trần là số hop; h có chỉ số dưới — h_v, h_q — là embedding.)

Lệch nhiều nghĩa là đang nhìn hai đường

Credit chia đều cho các hop *của riêng từng đường*. Nhưng một cạnh nằm trên hai đường thì nhận credit từ cả hai:

```
        e1            e2
   X --------> C -----------> D        pi_A   2 hop,  c_A = 0.30
              |
              | e3         e4
              +---> E ---------> D     pi_B   3 hop,  c_B = 0.60
```

```
bảng IG     e1  =  0.15 + 0.20  =  0.35      <- nằm trên cả hai đường
            e2  =  0.15
            e3  =  e4  =  0.20
                                tổng  =  0.90  =  c_A + c_B
```

Quy hoạch động trả về $\{e_1, e_3, e_4\}$ giá trị $0{,}75$, nhưng giá trị thật của π_B chỉ là $0{,}60$. Thừa $0{,}15$ — đúng bằng $c_A/2$, phần rò sang qua cạnh dùng chung. Áp Kiểm tra 5 với $h = 3$, tổng $0{,}75$: mỗi hop lẽ ra $0{,}25$.

```
e1  =  0.35    +40%     cạnh dùng chung, bị thổi lên
e3  =  0.20    -20%
e4  =  0.20    -20%     cạnh riêng, bị kéo xuống so với trung bình
```

Hệ quả thứ hai nối thẳng vào suff: bật đúng $\{e_1, e_3, e_4\}$ chỉ lấy lại $c_B = 0{,}60$, vì π_A cần e_2 mà e_2 đang tắt.

$$\text{tỷ phần IG} = \frac{0{,}75}{0{,}90} = 0{,}83, \qquad \text{suff} = \frac{0{,}60}{0{,}90} = 0{,}67, \qquad \text{chênh } 0{,}16 = \text{phần rò}$$

Không phải lỗi cần sửa mà là cấu trúc cần báo cáo đúng: suff hụt $\rightarrow$ vòng greedy §4.11 chạy tiếp $\rightarrow \pi_A$ hiện ra $\rightarrow$ báo cáo một cây phân nhánh tại C, không phải một đường.

Đọc độ lệch: hình dạng quan trọng hơn độ lớn

c/h giả định đóng góp của đường là tích thuần túy của các gate. Model thật còn số hạng khác và còn phi tuyến, nên nó là mốc lý tưởng chứ không phải đẳng thức đúng tới sai số máy. Phân biệt bằng **hình dạng** của độ lệch, không bằng biên độ:

```
0.11  0.14  0.17  0.21     tăng đều, giãn cách đều     ->  một đường thật
                                                          (chính là ví dụ §4.8)

0.35  0.20  0.20           một hop tách hẳn khỏi       ->  hai đường chồng nhau
                           cụm các hop bằng nhau
```

Hai ca lệch ngang nhau về biên độ ($\pm$30–40%) nhưng khác hẳn về hình dạng. Một hop đứng riêng bên trên một cụm đều nhau là chữ ký của cạnh dùng chung.

B. Ra đường đi — từ bảng số thành cấu trúc

Bảng số chưa phải lời giải thích. Phần này ghép nó thành chuỗi có thứ tự.

4.6 Bài toán: đọc ra chuỗi thông điệp

Mỗi tầng GNN là một vòng truyền tin: mọi cạnh cùng gửi thông điệp một lượt. §4.4 đã chấm xong — mỗi cặp (cạnh, tầng) có một điểm IG. Việc còn lại là đọc ra **chuỗi**: tầng 1 đi cạnh nào, tầng 2 cạnh nào, cho tới khi chạm D.

Một ví dụ cụ thể. Truy vấn *"vợ của đạo diễn phim X là ai?"*. Seed trích từ truy vấn là node `Phim-X`. Tầng A chỉ định đích D là đoạn nói về bà `Y`. Trong đồ thị:

```
Phim-X  --đạo diễn-->  Ông-Z  --vợ-->  D
```

Chuỗi cần đọc ra: *tầng 1 đi cạnh "đạo diễn", tầng 2 đi cạnh "vợ"*. Hai hop, đúng thứ tự. Đó vừa là lời giải thích, vừa là nhãn để dạy model lần sau đi đúng lối này.

Vì sao không lấy luôn mấy cạnh điểm cao nhất. Đồ thị thật có hàng nghìn cạnh và bảng IG chấm tất cả ở mọi tầng. Lấy top-k thì thường được một nhúm cạnh rời rạc:

```
X --0.18--> A        P --0.21--> Q        R --0.19--> D
seed                                                 đích
```

Ba con số to nhất, mà không mảnh nào nối X tới D. Và kể cả khi chúng có nối được, một *tập* cạnh vẫn không nói **thứ tự** — mà thứ tự chính là thứ multi-hop cần đọc ra.

Ràng buộc thứ tự không phải quy ước, nó là nhân quả. Cạnh thứ i của chuỗi phải được dùng ở đúng tầng i. Nghe như luật tùy tiện, nhưng:

```
tầng 1   Phim-X gửi tin cho Ông-Z    ->  Ông-Z giờ mới "biết" nó liên quan truy vấn
tầng 2   Ông-Z gửi tin cho D         ->  tin này mới có nội dung

nếu tầng 1 không gửi, thì sang tầng 2 Ông-Z vẫn rỗng,
và tin nó gửi cho D chẳng mang thông tin gì
```

Một cạnh điểm IG cao mà node nguồn chưa nhận được gì thì không truyền được gì. Chuỗi phải liền mạch theo thời gian.

Phát biểu gọn. Gọi *đường* π là dãy $(e_1, 1), (e_2, 2), \ldots, (e_h, h)$ nối đuôi nhau, e_1 bắt đầu ở một seed thuộc V_q và e_h kết thúc ở D; giá trị của nó là $\text{val}(\pi) = \sum_i \text{IG}_{e_i}^{(i)}$. Cần tìm π^* có val lớn nhất.

Đây chính là khái niệm *circuit* trong mech-interp — đồ thị con nhỏ nhất, liên thông, đủ tái tạo hành vi đang xét. Khác ở chỗ nó chạy trên đồ thị tri thức chứ không trên attention head, và trọng số do IG cấp.

Số đường từ seed tới D lớn theo hàm mũ nên không duyệt hết được. Hai mục sau tìm π^* trong **một lượt quét**: §4.7 đổi cách vẽ đồ thị, §4.8 quét.

4.7 Vẽ lại đồ thị: mỗi tầng một bản sao

Ràng buộc *"cạnh thứ i ở tầng i"* khó cài trên đồ thị gốc, vì cùng một cạnh dùng được ở nhiều tầng với điểm khác nhau — mà trên đồ thị gốc không có chỗ nào ghi "tầng mấy". Cách gỡ: **vẽ mỗi node một lần cho mỗi tầng.**

Hình dung một bảng — hàng là node, cột là tầng. Ô (v, l) là trạng thái của v sau l vòng truyền tin:

```
          tầng 0     tầng 1     tầng 2     tầng 3
  X         *          .          .          .
  C         .          .          .          .
  D         .          .          .          #
          * seed                            # đích
```

Mỗi cạnh thật $e = (v', r, v)$ dùng ở tầng l thành một mũi tên từ ô $(v', l-1)$ sang ô (v, l) — **luôn đi sang phải đúng một cột**, mang trọng số $\text{IG}_e^{(l)}$. Thêm một loại mũi tên nữa, *đứng yên*, từ $(v, l-1)$ sang (v, l) trọng số 0, để một node giữ nguyên trạng thái qua một tầng:

$$\widehat{V} = V \times \{0, 1, \ldots, L\}, \qquad \widehat{\mathcal{E}} = \underbrace{\{((v', l-1), (v, l))\}}_{\text{cạnh thật, trọng số } \text{IG}_e^{(l)}} \cup \underbrace{\{((v, l-1), (v, l))\}}_{\text{cạnh đứng yên, trọng số } 0}$$

Ba điều rơi ra ngay. **Một**, ràng buộc tầng biến mất — nó đã nằm sẵn trong cấu trúc, mọi đường trên đồ thị này tự động dùng cạnh thứ i ở đúng tầng i. **Hai**, không có chu trình, vì mọi

mũi tên đi sang phải đúng một cột; quét một lượt theo thứ tự cột là xong, không cần Dijkstra. **Ba**, đích là ô (D, L) chứ không phải $(D, \text{bất kỳ})$, vì $p(D)$ tính từ $h_D^{(L)}$.

Trọng số 0 của cạnh đứng yên không phải lựa chọn thiết kế: gate chỉ gắn trên thông điệp, mà đứng yên thì không có thông điệp nên không có IG để tính. Nó tồn tại để đường ngắn hơn L hop vẫn sống tới được cột L.

4.8 Quét một lượt (quy hoạch động)

Ý tưởng gói trong một câu: **đi từ trái sang phải, tới mỗi ô chỉ giữ lại lối tốt nhất đến ô đó.**

Đứng ở ô (v, l) mà hỏi *"có những cách nào tới đây?"* thì chỉ có hai loại — v đứng yên từ $(v, l-1)$, hoặc một cạnh nào đó vừa đổ vào từ $(v', l-1)$. Cả hai đều xuất phát từ **cột** $l-1$, mà cột đó đã tính xong. Nên chỉ việc so vài con số, giữ cái lớn nhất, và ghi lại nó đến từ đâu:

$$\text{best}[(v,0)] = \begin{cases} 0 & v \in V_q \quad \text{(mọi seed, không chỉ một)} \\ -\infty & \text{ngược lại} \end{cases}$$

$$\text{best}[(v,l)] = \max\Big\{ \underbrace{\text{best}[(v,l-1)]}_{\text{đứng yên, } +0}, \quad \max_{e=(v',r,v)} \underbrace{\text{best}[(v',l-1)] + \text{IG}_e^{(l)}}_{\text{đi chéo}} \Big\}$$

Không bao giờ phải quay lại cột đã qua — đó là lý do một lượt quét là đủ, và vì sao chi phí chỉ là $O\big((|V| + |\mathcal{E}|) \times L\big)$.

Lưu kèm back$[(v,l)]$ — cách nào thắng — để truy ngược. Giá trị $-\infty$ nghĩa là chưa có tín hiệu nào tới được ô này; cộng gì vào $-\infty$ cũng vẫn $-\infty$, đó là cơ chế tự động chặn các cạnh nổ quá sớm.

Nhiều seed chỉ đổi dòng khởi tạo. QHĐ đa nguồn tự tìm đường tốt nhất xuất phát từ bất kỳ seed nào, và truy ngược cho biết nó từ seed nào. Chi phí $O\big((|V| + |\mathcal{E}|) \times L\big)$, một lượt quét.

$\sigma(D)$ **— mấy seed với tới được đích.** Miễn phí cùng lượt quét: thêm vào mỗi ô một bitmask rộng $|V_q|$ ghi seed nào với tới được ô đó.

```
reach[(v,0)]  =  {s}   nếu v = s thuộc V_q,    rỗng  nếu không

reach[(v,l)]  =  reach[(v, l-1)]                <- đứng yên
                 hợp  U reach[(v', l-1)]        <- mọi cạnh e=(v',r,v)

sigma(D)  =  | reach[(D,L)] |
```

Khác best ở chỗ best lấy *max* — một người thắng — còn reach lấy *hợp*: mọi seed với tới được, không riêng seed nằm trên đường tốt nhất. Chi phí một từ máy mỗi ô khi $|V_q| \le 64$, phép hợp là OR bit, không thêm lượt quét nào. Nhất quán với tiền kiểm §4.11: $\sigma(D) = 0 \iff \text{best}[(D,L)] = -\infty$.

$\sigma(D)$ **không vào công thức nào** — nó là đại lượng chẩn đoán, dùng ở §6.1 để tách hai ca "bằng chứng phân tán" khác hẳn nhau.

Chạy tay.

```
X --e1--> C --e2--> D7 --e3--> B --e4--> D15       V_q={X}, L=4

bảng IG cho đích D15:
cạnh          tầng1   tầng2   tầng3   tầng4
e1  X->C       0.11    0.01    0.00    0.00
e2  C->D7      0.00    0.14    0.01    0.00
e3  D7->B      0.00    0.00    0.17    0.01
e4  B->D15     0.00    0.00    0.00    0.21
```

Để ý hình chéo: mỗi cạnh chỉ có giá trị ở đúng một tầng. Đó là chữ ký của một chuỗi sạch, và nó đọc ra luôn thứ tự hop.

```
cột 1   C    đứng yên -inf | e1: best[(X,0)]+0.11 = 0.11     -> 0.11 từ (X,0)
        D7   đứng yên -inf | e2: best[(C,0)]+0.00 = -inf      -> -inf
                                    ^ C chưa nhận gì ở cột 0

cột 2   C    đứng yên 0.11 thắng | e1 tầng2: 0+0.01 = 0.01    -> 0.11
        D7   đứng yên -inf | e2: best[(C,1)]+0.14 = 0.25      -> 0.25 từ (C,1)

cột 3   D7   đứng yên 0.25 thắng | e2 tầng3: 0.11+0.01 = 0.12 -> 0.25
        B    đứng yên -inf | e3: best[(D7,2)]+0.17 = 0.42     -> 0.42 từ (D7,2)

cột 4   B    đứng yên 0.42 thắng | e3 tầng4: 0.25+0.01 = 0.26 -> 0.42
        D15  đứng yên -inf | e4: best[(B,3)]+0.21 = 0.63      -> 0.63 từ (B,3)
```

```
          tầng0   tầng1   tầng2   tầng3   tầng4
  X         0       0       0       0       0
  C       -inf    0.11    0.11    0.11    0.11
  D7      -inf    -inf    0.25    0.25    0.25   #
  B       -inf    -inf    -inf    0.42    0.42
  D15     -inf    -inf    -inf    -inf    0.63   #
```

Ranh giới giữa $-\infty$ và số hữu hạn chạy chéo xuống — đó là mặt sóng thông tin, cho thấy tín hiệu lan tới đâu sau mỗi tầng.

4.9 Trích đường — max-sum, không phải min-cost

$$\pi_D^* = \underset{\pi:(V_q,0)\to(D,L)}{\operatorname{argmax}} \sum_{(e,l)\in\pi} \mathrm{IG}_e^{(l)} \qquad = \qquad \text{truy ngược back[] từ ô } (D,L)$$

Phản xạ thông thường là đặt cost $= -\log w$ rồi tìm đường rẻ nhất, như Dijkstra trên đồ thị xác suất. **Ở đây làm vậy là sai loại.** $-\log$ rồi cộng tương đương với *nhân* các trọng số — hợp lý khi w là xác suất của những sự kiện độc lập. Nhưng IG là một phân rã *cộng tính*: $\sum$IG bằng đúng cái bánh (§4.4, Kiểm tra 4). Đem phân rã cộng tính xử lý như nhân tính là lệch ngay từ gốc; thiên vị độ dài dưới đây chỉ là triệu chứng dễ đo nhất.

Triệu chứng: thiên vị đường ngắn, phạt hai lần

Trọng số đã chuẩn hóa nên $w \le 1$, kéo theo $-\log w \ge 0$ — mỗi cạnh thêm vào luôn cộng một khoản dương. Đồng thời §4.10 nói mỗi cạnh chỉ nhận c/h, nên đường dài hơn có trọng số mỗi cạnh **nhẹ hơn**, tức $-\log$ của nó **lớn hơn**. Hai hiệu ứng nhân lên nhau. Hai đường giá trị hoàn toàn bằng nhau, $c = 0{,}60$:

```
                      max-sum                      min(-log)
A   2 hop    0.30 + 0.30          = 0.60      2 x 1.20  =   2.41
B   6 hop    6 x 0.10             = 0.60      6 x 2.30  =  13.82

max-sum    : hòa -- đúng, chúng đáng giá như nhau
min(-log)  : A thắng gấp 5.7 lần -- sai
```

Viết thành công thức thì hai khoản phạt hiện ra tách bạch:

$$\mathrm{cost}(h) \;=\; h\cdot\big(-\log(c/h)\big) \;=\; \underbrace{h\,(-\log c)}_{\text{phạt 1: do số cạnh}} \;+\; \underbrace{h\log h}_{\text{phạt 2: do } c/h \text{ pha loãng}}$$

Điều này chí mạng ở đây vì **đường dài chính là thứ ta muốn tìm** — multi-hop là lý do tồn tại của cả tầng B. Một hàm mục tiêu thiên vị đường ngắn sẽ chọn lối tắt giả 2 hop và bỏ chuỗi suy luận thật 4 hop.

Và max-sum còn được thêm hai thứ: không cần ReLU để bỏ cạnh âm — nó tự né mà vẫn giữ được thông tin *"cạnh này đẩy ngược"*; và giá trị đường đọc thẳng ra phần trăm, so trực tiếp được với ngưỡng suff. Max-sum trên DAG vẫn tuyến tính — longest-path chỉ khó trên đồ thị có chu trình.

Truy ngược ví dụ trên:

```
(D15,4) <- e4 tầng4 <- (B,3) <- e3 tầng3 <- (D7,2) <- e2 tầng2 <- (C,1)
        <- e1 tầng1 <- (X,0)

giá trị  0.11 + 0.14 + 0.17 + 0.21 = 0.63      khớp ô đích
```

C. Kiểm chứng và ca khó

Đường tìm được phải qua một phép thử thật, rồi mới tới các ca rắc rối.

4.10 Kiểm chứng (bắt buộc)

Che gate theo Π rồi chạy một lượt. Gọi g^* là điểm cuối của lần chạy IG đang xét (§4.4) — $\hat{g}$ khi báo cáo, 1 khi đào nhãn — và đặt $g_\Pi[e,l] = g^*[e,l]$ nếu $(e,l) \in \Pi$, ngược lại 0. Cả hai đại lượng chuẩn hóa theo cái bánh $F_B(g^*) - F_B(0)$ nên nằm trong $[0,1]$:

$$\text{suff}(\Pi) = \frac{F_B(g_\Pi) - F_B(0)}{F_B(g^*) - F_B(0)}, \qquad \text{nec}(\Pi) = \frac{F_B(g^*) - F_B(g_{\neg\Pi})}{F_B(g^*) - F_B(0)}$$

$$\text{nhận } \Pi \iff \text{suff}(\Pi) \geq \theta_s, \qquad \theta_s = 0{,}95$$

Cả chuỗi phía sau — bảng IG, QHĐ, suff, cái bánh, $\tilde{\varepsilon}$ — dùng **cùng một** g^*. Hai lần chạy ở §4.4 là hai lượt độc lập; đừng trộn số của lượt này vào lượt kia.

suff là phần cái bánh mà Π lấy lại được; nec là phần mất đi khi tắt Π. Chuẩn hóa là bắt buộc chứ không phải cho đẹp: F_B là log xác suất nên âm, so trực tiếp suff $\geq \theta_s \cdot F_B(g^*)$ sẽ đòi suff tốt hơn cả khi bật đủ gate — điều kiện không bao giờ thỏa.

nec ≈ 1 nghĩa là Π thiết yếu; nec ≈ 0 nghĩa là có đường song song, một phát hiện đáng ghi chứ không phải lỗi. IG là bộ sàng, không phải bằng chứng; bước này không bỏ được — và nó là vai thứ ba trong một chuỗi ba vai tách bạch:

```
IG     chấm điểm từng (cạnh, tầng)       ->  một bảng số, chưa phải đường
       3M ~ 60 lượt, bất kể bao nhiêu cạnh      đề xuất

QHĐ    nối các điểm đó thành đường       ->  đường đi
       O((|V|+|E|)*L), không gọi model          dựng hình

suff   bật đúng đường đó, chạy model     ->  nhận hoặc bỏ
       1 lượt forward mỗi lần                   phán quyết
```

Lý do phải tách: IG nói *"nếu vặn mạnh các cạnh này thì $p(D)$ tăng"* — một phát biểu về **đạo hàm**. Nó không nói *"chỉ riêng các cạnh này là đủ"* — đó là phát biểu về **giá trị hàm tại một điểm khác**. Model phi tuyến nên hai điều đó lệch nhau.

> **Đừng nhầm** suff **với tỷ phần IG.** Tỷ phần IG ($\sum$IG trên đường chia tổng IG) là con số *suy ra* từ phép phân rã cộng tính; suff là con số *đo thật* khi bật đúng các gate đó rồi chạy model. Chúng khác nhau vì model phi tuyến — một đường chiếm 68% tỷ phần IG có thể chỉ đạt suff $= 0{,}61$. Chỉ suff mới quyết định nhận hay không. Xem thêm §10 của phụ lục *Những chỗ dễ sai*.

4.11 Hội tụ nhiều seed — greedy xóa cạnh, rồi tỉa ngược

Khi một đích cần nhiều lối đi, một đường không đủ. Không cần Steiner tree — một tiền kiểm và hai pha, tất cả đều dùng lại QHĐ §4.8.

Tiền kiểm: trần của suff

Luôn tồn tại tập cho suff $= 1$ — bật hết mọi gate. Nhưng ta chỉ được dùng *đường xuất phát từ* V_q, mà cái bánh thì gồm cả đóng góp của những cạnh không đường nào chạm tới:

```
V_q = {X}.  Node Z không với tới được từ X trong L hop, nhưng có cạnh Z -> D ở tầng 3.

g = 0   thông điệp đó không chảy
g = g^  nó chảy, làm đổi h_D và kéo theo p(D)

=>  cạnh đó có trong cái bánh   (mẫu số)
=>  nhưng QHĐ không bao giờ đưa nó vào đường nào, vì best[(Z,2)] = -inf   (tử số)
```

h_Z không mang thông tin truy vấn, nhưng vẫn là nội dung văn bản và vẫn xê dịch điểm số. Phần *credit mồ côi* đó chặn suff ở một trần nhỏ hơn 1. Trần này đọc miễn phí từ bảng QHĐ: gọi $E_{\text{reach}} = \{(e,l) : \text{best}[(v', l-1)] > -\infty\}$ là tập cạnh có thể nằm trên một đường nào đó, thì

$$\text{suff}_{\max} \;=\; \frac{\sum_{(e,l)\in E_{\text{reach}}} \text{IG}_e^{(l)}}{\sum_{(e,l)} \text{IG}_e^{(l)}}, \qquad \text{suff}_{\max} < \theta_s \;\Rightarrow\; \text{dừng ngay.}$$

Kết luận lúc đó *không phải* lỗi đồ thị mà là: lối đúng không xuất phát từ seed. Nguyên nhân thường gặp là V_q trích thiếu thực thể — sửa ở khâu entity linking, không phải khâu xây graph.

Thuật toán

```
tiền kiểm   nếu suff_max < theta_s:  dừng, ghi "không giải thích được bằng đường từ seed"

-- chặng 1: gom ------------------------------------------
Pi_D <- rỗng
lặp:
    pi   = truy ngược back[] từ (D,L)
    gain = tổng IG của các cạnh mới trong pi
    Pi_D <- Pi_D + pi

    nếu suff(Pi_D) >= theta_s   ->  dừng:  thành công
    nếu gain < eps              ->  dừng:  phân tán   (eps = eps_tilde*cake)
    nếu |Pi_D| = n_max          ->  dừng:  hết ngân sách đọc hiểu

    đặt IG_e^(l) = 0 cho mọi (e,l) thuộc Pi_D      # đã trả tiền rồi
    chạy lại QHĐ §4.8

-- chặng 2: tỉa ngược ------------------------------------
cho mỗi pi trong Pi_D (theo thứ tự đã thêm):
    nếu suff(Pi_D \ {pi}) >= theta_s:
        Pi_D <- Pi_D \ {pi}               # đường thừa, bỏ
```

$n_{\max}$ không phải hằng chọn bừa — nó là $\lceil \theta_s/\tilde{\varepsilon} \rceil$

Tham số gốc là $\tilde{\varepsilon}$: **một đường phải mang ít nhất bao nhiêu phần cái bánh mới đáng báo cáo**, tức $\tilde{\varepsilon} = \varepsilon/[F_B(g^*) - F_B(0)]$, mặc định $0{,}10$. Từ đó $n_{\max}$ suy ra chứ không chọn. Mỗi đường được

nhận có $\text{gain}_i \geq \varepsilon$; các gain rời nhau vì cạnh đã lấy bị đặt IG $= 0$ nên mỗi cạnh chỉ vào gain của đúng một đường; và vòng lặp còn chạy nghĩa là tích lũy chưa đạt $\theta_s \cdot \text{cake}$. Vậy $(n-1)\varepsilon \leq \sum_{i<n} \text{gain}_i < \theta_s \cdot \text{cake}$, kéo theo

$$n \;\leq\; \left\lceil \frac{\theta_s}{\tilde{\varepsilon}} \right\rceil \;=\; \left\lceil \frac{0{,}95}{0{,}10} \right\rceil \;=\; 10.$$

Nên ε và $n_{\max}$ là *một núm, không phải hai*. Hằng $n_{\max} = 5$ tương đương $\tilde{\varepsilon} \approx 0{,}19$ — từ chối mọi đường mang dưới 19% cái bánh, hà khắc hơn nhiều so với vẻ ngoài của nó.

Chặn dùng $\sum \text{gain} \leq \text{cake}$, đúng khi mọi cạnh trên đường có IG dương. Nếu đồ thị có cạnh IG âm thì mẫu số phải là $\sum \text{IG}^+$ và chặn nới ra chút; QHĐ max-sum vốn né cạnh âm nên hiếm gặp.

Vòng lặp luôn dừng, kể cả khi bỏ hết trần

Mỗi đường được nhận phải mang ít nhất một cạnh mới — cạnh cũ có IG $= 0$ nên không sinh gain. Số cạnh với tới được là hữu hạn, nên $n \leq |E_{\text{reach}}|$. Chặn này vô dụng về mặt số nhưng bảo đảm thuật toán không chạy vô hạn dù tháo hết mọi ngưỡng.

Hai chặn đã thử và loại

```
2|V_q|  (hay |V_q| + k)
    số seed không chặn được phân nhánh -- một seed vẫn tách nhiều lối (hình dưới).
    Seed thuộc về sàn: nếu bằng chứng đến từ k seed thì cần ít nhất k đường.
    Đặt nó làm trần là lỗi của |V_q| + 1 ở bản trước.

mincut (Menger)
    số đường rời cạnh tối đa = mincut, đúng và tính được -- nhưng đường của ta
    không rời cạnh:
        X->A->D  và  X->A->E->D   dùng chung thân X->A,  mincut = 1
    mincut đếm lối vào độc lập, không đếm đường cần thiết.
    Ép rời cạnh cũng không cứu được: ca hội tụ ở Ví dụ A dùng chung đuôi F2->D7
    -- ép rời là giết đúng trường hợp thuật toán sinh ra để xử lý.
```

```
       A
      /   \
X -> B --> D          một seed, ba lối, cả ba đều cần
      \   /
       C
```

Ba lý do dừng, ba chẩn đoán khác nhau

```
suff đạt theta_s     thành công

gain biên < eps      không lối nào mang đủ, bằng chứng rải mỏng
                     đọc kèm sigma(D) của §4.8 để biết phân tán kiểu gì:
                        sigma cao -> nhiều nguồn thật, mỗi nguồn nhỏ -- bằng chứng vững
                        sigma = 1 -> một nguồn duy nhất và mong manh -- đáng ngờ

|Pi_D| = n_max       cần hơn n_max lối độc lập mới đủ
                     -> không phải thất bại: đồ thị có nhiều nguồn bằng chứng song
                        song cho D, bản thân nó là thông tin đáng ghi
```

Pha 1 — vì sao xóa cạnh

```
cạnh mới    vẫn có giá trị  ->  đường kế tiếp được thưởng vì mang thêm cái mới
cạnh đã có  giá trị 0       ->  dùng lại miễn phí, không bị tính hai lần
```

`gain` ở trên đo đúng phần mới đó, nên nó tụt tự nhiên khi các đường còn lại chỉ dùng lại cạnh cũ.

Pha 2 — vì sao cần

Greedy luôn lấy đường đơn giá trị cao nhất trước, mà đường đó chưa chắc thuộc tập tối thiểu:

```
ba nhánh A, B, C cùng vào một node
A + B cùng nhau là đủ; C một mình giá trị cao nhất nhưng vô dụng nếu thiếu A,B

vòng 1 chọn C  ->  suff thấp
vòng 2 chọn A  ->  suff thấp
vòng 3 chọn B  ->  suff đạt
-> báo cáo thừa nhánh C
```

Pha 2 phát hiện và loại C. Chi phí $|\Pi_D|$ lượt forward, thường 2–3. Cùng tinh thần với vòng đi biên §3.4: chọn dần từ rỗng rồi dọn lại, đừng tin greedy là xong.

Ví dụ A — hội tụ ngay tầng kế tiếp

```
X --đạo_diễn--+
              +--> F2 --included_in--> D7
Y --đóng------+

IG:  X->F2 tầng1 = 0.22,  Y->F2 tầng1 = 0.19,  F2->D7 tầng2 = 0.33
     tổng toàn đồ thị = 0.81   (0.07 rải ở cạnh nhiễu)
```

```
vòng 1   best[(F2,1)] = max(0.22 từ X, 0.19 từ Y) = 0.22
         best[(D7,2)] = 0.22 + 0.33 = 0.55        -> pi_1 qua X
         suff = 0.61  <  0.95      chưa đủ

vòng 2   xóa (X->F2,1) và (F2->D7,2)
         best[(F2,1)] = 0.19 từ Y
         best[(D7,2)] = 0.19 + 0.00 = 0.19        -> pi_2 qua Y
                               ^ cạnh dùng chung giờ miễn phí

         hợp cạnh: 0.22+0.19+0.33 = 0.74 / 0.81 = 91%
         suff = 0.97    đủ

chặng 2  bỏ pi_1 -> suff 0.29 ;  bỏ pi_2 -> suff 0.61    cả hai < 0.95
         -> giữ cả hai
```

Phép cộng khớp hai cách — đó là hệ quả của việc xóa cạnh:

$$\underbrace{0{,}55}_{\pi_1 \text{ đầy đủ}} + \underbrace{0{,}19}_{\pi_2 \text{ biên}} = 0{,}74 = \text{hợp cạnh}$$

Kết quả là một cây hội tụ, tìm ra bằng đúng vòng lặp đường đi:

```
  X --0.22--+
            +--> F2 --0.33--> D7
  Y --0.19--+
  tầng1           tầng2
```

Ví dụ B — hội tụ ở tầng sâu

Độ sâu không đổi gì trong thuật toán: xóa cạnh làm toàn bộ đuôi dùng chung thành miễn phí, bất kể đuôi dài 1 hay 5 cạnh.

```
X --> A --> B --+
                +--> M --> D        hội tụ ở M, tầng 3
Y --> C --> E --+

X->A t1 0.11   A->B t2 0.09   B->M t3 0.12
Y->C t1 0.10   C->E t2 0.08   E->M t3 0.10      M->D t4 0.10
tổng toàn đồ thị = 0.76
```

```
vòng 1   best[(A,1)]=0.11   best[(C,1)]=0.10
         best[(B,2)]=0.20   best[(E,2)]=0.18
         best[(M,3)]= max(0.20+0.12=0.32 qua B, 0.18+0.10=0.28 qua E) = 0.32
         best[(D,4)]= 0.32+0.10 = 0.42           -> pi_1 = X->A->B->M->D
         suff = 0.58  <  0.95      chưa đủ

vòng 2   xóa cả bốn cạnh của pi_1
         best[(B,2)] = 0 + 0 = 0                  nhánh X giờ vô giá trị
         best[(M,3)] = max(0, 0.18+0.10=0.28) = 0.28 qua E
         best[(D,4)] = 0.28 + 0 = 0.28            -> pi_2 = Y->C->E->M->D
                               ^ M->D miễn phí

         hợp cạnh: 0.70 / 0.76 = 92%
         suff = 0.96    đủ
```

Điểm hội tụ đọc được miễn phí

```
pi_1 = (X,0) (A,1) (B,2) (M,3) (D,4)
pi_2 = (Y,0) (C,1) (E,2) (M,3) (D,4)
                          \___/
               ô chung có l nhỏ nhất = điểm hội tụ
```

So hai danh sách ô, lấy ô chung có l nhỏ nhất. Không cần thuật toán riêng.

Cảnh báo: tỷ phần IG không đo được tầm quan trọng của nhánh

Theo tính chất c/h ở §4.5, một cấu trúc gồm h cạnh cùng cần thiết chia đều c cho cả h. Nên tỷ phần của một nhánh phản ánh số cạnh của nó, không phải tầm quan trọng:

```
nhánh X  4 cạnh  ->  0.42  =  55%
nhánh Y  3 cạnh  ->  0.28  =  37%     nhưng hai nhánh thiết yếu ngang nhau
```

Cực đoan hơn:

```
X --> A --> B --> C --> M --> D        5 cạnh  ->  83% tỷ phần IG
Y -------------------> M               1 cạnh  ->  17% tỷ phần IG

bỏ Y đi thì D chết hệt như bỏ X đi
```

Càng sâu, hai nhánh càng dễ lệch độ dài, tỷ phần IG càng dễ gây hiểu nhầm.

```
Sai    so hai nhánh bằng tổng IG       (bị số cạnh chi phối)
Đúng   so bằng mức tụt của suff khi bỏ nhánh đó đi
```

Phép đo đúng tốn 1 lượt forward mỗi nhánh, và chặng 2 đã chạy sẵn rồi:

```
bỏ nhánh X  ->  suff tụt về 0.31
bỏ nhánh Y  ->  suff tụt về 0.35      -> thiết yếu ngang nhau
trong khi tỷ phần IG nói 55% / 37%
```

Thay thế cho chặng 1. Giữ k giá trị tốt nhất mỗi ô (k-best), chi phí $O((|V| + |\mathcal{E}|) \cdot L \cdot k)$. Nhanh hơn, nhưng vòng lặp xóa cạnh xử lý hội tụ tự nhiên hơn vì cạnh dùng chung tự thành miễn phí.

4.12 Nhiều đích — chia chung lượt forward

F_B khác nhau theo đích nên không gộp được — gộp thành $\sum_D \log p(D)$ thì rẻ hơn nhưng mất khả năng biết cạnh nào phục vụ đích nào. Tiết kiệm bằng cách dùng chung forward:

```
tại mỗi alpha:   1 forward  ->  |T| lần backward trên cùng đồ thị đó

M=20, |T|=6:   20 forward + 120 backward
     thay vì   120 forward + 120 backward
```

5 Nhiều đích và quan hệ phụ thuộc

Một đích có thể nằm trên đường tới đích khác:

```
X --sinh_ở--> C --inc_in--> D7 --mentions--> B --thuộc--> D15
                            |
                      D7 vừa là đích, vừa là trạm trung chuyển
```

Xảy ra được vì QuadGraph có cạnh liên tầng hai chiều: entity → document, rồi document → entity khác. Đây chính là multi-hop ở mức tài liệu.

5.1 Quan hệ này không bao giờ vòng

D_7 tới được ở tầng 2, D_{15} ở tầng 4 qua D_7. Không có cách nào D_{15} lại nằm trên đường tới D_7, vì chỉ số tầng chỉ tăng. Nên quan hệ phụ thuộc giữa các đích tự nó là một DAG, và tầng-tới-được cho sẵn thứ tự topo.

5.2 Hai thứ rơi ra miễn phí từ bảng QHĐ

$$\text{depth}(D) = \min\{\, l : \text{best}[(D, l)] > -\infty \,\}$$

tức cột đầu tiên mà ô hết $-\infty$. Trong bảng ở §4.8: $\text{depth}(D_7) = 2$, $\text{depth}(D_{15}) = 4$. Không cần lượt quét riêng.

PHÁT HIỆN TRẠM D là trạm của D' $\iff$ truy ngược của D' chạm ô (D, l) nào đó

Trong ví dụ §4.9, truy ngược D_{15} đi xuyên qua ô $(D_7, 2)$ — mà $D_7 \in T$. Phát hiện quan hệ phụ thuộc không cần thuật toán riêng, nó lộ ra ngay trong lúc dựng đường.

5.3 Giải theo thứ tự nông → sâu

```
-- bước 1: thứ tự ------------------------------------------
sắp T theo depth(D) tăng dần        (đọc từ bảng QHĐ, miễn phí)

-- bước 2: giải lần lượt -----------------------------------
Ecov <- rỗng                        # cạnh đã nằm trong báo cáo

cho mỗi D theo thứ tự depth:
    IG_D  <- chạy IG với F_B = log p(D)
    IG_D[e,l] <- 0  với mọi (e,l) thuộc Ecov   # chỉ cho mục tiêu tìm kiếm
    chạy QHĐ + greedy  ->  Pi_D
    Ecov <- Ecov + cạnh(Pi_D)
```

```
-- D7  (depth 2) -------------------------------
đường X -> C -> D7            0.52 / 0.60 = 87%
Ecov = { (X->C,1), (C->D7,2) }

-- D15 (depth 4) -------------------------------
bảng IG của D15 sau khi xóa:
    X->C    tầng1   0.11 -> 0.00   đã có
    C->D7   tầng2   0.14 -> 0.00   đã có
    D7->B   tầng3   0.17
    B->D15  tầng4   0.21
                    ----- mới = 0.38
```

Báo cáo thành một cây, không phải hai đường rời:

```
X --0.24--> C --0.28--> D7 #          87% của D7
                        |
                        +--0.17--> B --0.21--> D15 #     bổ sung 0.38
```

Trung thực hơn hẳn: báo riêng thì hai cạnh đầu bị tính hai lần, người đọc tưởng có hai chuỗi bằng chứng trong khi chỉ có một chuỗi phân nhánh.

5.4 Chỗ dễ sai: suff tính trên toàn đường

```
xóa cạnh  ->  chỉ để làm mục tiêu tìm kiếm và báo cáo giá trị biên
suff      ->  luôn tính trên toàn bộ cạnh của đường, kể cả tiền tố

Sai:   suff(D15) = bật chỉ { D7->B, B->D15 }
Đúng:  suff(D15) = bật cả  { X->C, C->D7, D7->B, B->D15 }
```

Bật thiếu tiền tố thì D_7 không có tín hiệu, B cũng không, $p(D_{15})$ tụt — bạn sẽ kết luận nhầm là đường sai.

5.5 Tiền tố có thể khác nhau giữa các bảng IG

Bảng IG của D_{15} và của D_7 là hai bảng khác nhau (hai F_B khác nhau): trong bảng của D_{15}, đường tới D_7 có giá trị $0{,}25$; trong bảng của D_7 nó là $0{,}52$. Thường cấu trúc trùng, chỉ khác độ lớn. Nếu tiền tố khác nhau thì đó là thông tin thật:

```
cùng tiền tố   ->  một chuỗi duy nhất, D7 thật sự là trạm trung chuyển
khác tiền tố   ->  có hai lối tới D7; lối tốt cho D7 khác lối tốt để đi tiếp
                       -> báo cáo cả hai, đừng gộp
```

Khi kiểm tra quan hệ phụ thuộc, so tập cạnh, đừng chỉ nhìn node.

5.6 Đánh đổi

Thuật toán này phụ thuộc thứ tự: giải D_{15} trước có thể ra lối trực tiếp khác không qua D_7. Sắp theo depth là lựa chọn có cơ sở (khớp chiều thông tin chảy) nhưng là greedy, không tối ưu toàn cục. Tối ưu thì thành Steiner tree nhiều đích, NP-khó. Với $|T|$ cỡ 5–8 thì greedy theo depth là đủ, và cái bạn được lại là cấu trúc đọc hiểu được.

6 Nối hai tầng và train

$$\text{nhãn mỗi ví dụ} = \Big(q,\ T,\ \{\Pi_D\}_{D\in T},\ \{\text{depth}(D)\},\ \{w(D)\}\Big)$$

Chỉ tập T đi qua ranh giới hai tầng; không điểm số nào đi qua — xem §1 của phụ lục *Những chỗ dễ sai*.

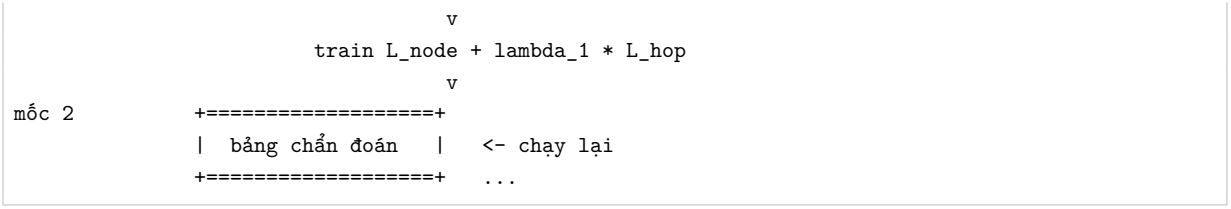
Ký hiệu mục này giới thiệu.

```
w(D)             trọng số theo quan hệ phụ thuộc giữa các đích
S_k              tập ví dụ "lỗi model" ở mốc k -- cũng là tập nhãn
rank_GFM(D)      vị trí của node tài liệu D khi sắp p(v) giảm dần
k_top(q)         ngưỡng hạng, theo từng ví dụ: ceil(beta*|T(q)|), beta ~ 1.5
k_infer          ngưỡng top-k lúc chạy thật -- dùng để báo cáo, không để lọc
V_neg            node âm tính lấy mẫu (dùng chung cho L_node và L_hop)
V_hop(l)         miền cộng của L_hop ở tầng l (§6.3)
lambda_1,2       hệ số của L_hop và L_edge
R(T), R(V_q)     tập node với tới được từ T / từ V_q trong vài hop
```

6.1 Bảng chẩn đoán — chạy ở mỗi mốc

Bảng này **không chạy một lần rồi thôi**, và nó không nằm trước hay sau việc train — nó nằm *giữa*, và lặp lại ở mỗi mốc (§6.5):

```
mốc 0    đào T  -->  train L_node tới hội tụ
                      |   model đã cố hết sức với nhãn rẻ nhất
                      v
mốc 1          +====================+
               |  bảng chẩn đoán    |   <- chạy tầng B ở đây
               +=========+==========+
                 +-------+---------+
      đọc cách 1 |                 | đọc cách 2
 "tiến độ tới đâu?"           "lấy nhãn ở đâu?"
  phân bố các ô               ô «lỗi model» = S_1
                                      |
```



Một lần chạy, hai cách đọc: **báo cáo về đợt train vừa xong**, và **nguồn nhãn cho đợt tới**.

Lấy số từ đâu

Bảng không phải một thủ tục riêng: nó là đầu ra của tầng B đọc theo một góc khác. Bốn đại lượng, cả bốn rơi ra từ một lần chạy tầng B:

đại lượng	lấy từ đâu	chi phí
rank_GFM(D)	1 lượt forward, đọc p(v) của mọi node	rẻ
best[(D,L)]	bảng QHĐ §4.8 -- hữu hạn hay -inf?	miễn phí
suff(Pi_D)	kiểm chứng §4.5	1 lượt mỗi đường
|Pi_D|, độ dài	truy ngược	miễn phí
sigma(D)	bitmask reach §4.8	miễn phí

Hai điều kiện chính đo *hai thứ độc lập nhau* — $\text{suff}(\Pi_D)$ hỏi đồ thị có chứa lối đi kiểm chứng được không, $\text{rank}_{\text{GFM}}(D)$ hỏi model có thật sự dùng lối đó không — nên xếp thành hai trục chứ không phải một danh sách.

Tiền kiểm — chạy trước, vì chúng loại hẳn ví dụ ra khỏi bảng chính:

Điều kiện	Chẩn đoán	Hành động
$\lvert T(q)\rvert > k_{\text{infer}}$	lệch cấu hình	nới k_{infer}, hoặc hạ θ ở §3.3
$\text{best}[(D, L)] = -\infty$	lỗi cấu trúc	tăng L, hoặc nối lại đồ thị
$\text{suff}_{\text{max}} < \theta_s$	lối đúng không xuất phát từ seed	sửa entity linking (V_q thiếu), hoặc chấp nhận giới hạn phương pháp

Bảng chính — hai trục:

	rank $\le k_{\text{top}}(q)$ model lấy được D	rank $> k_{\text{top}}(q)$ model bỏ sót D
suff $\ge \theta_s$	ổn — không cần làm gì	**lỗi model** $\rightarrow$ tập S_k; đường tồn tại mà model không đi, Π và $\widehat{w}$ của nó là nhãn train
suff $< \theta_s$	nghi **đường tắt** — model đúng mà không lối nào giải thích nổi	**lỗi đồ thị** — bổ sung cạnh lúc xây graph

Ô *lỗi model* vừa là một chẩn đoán, vừa là tập nhãn. Chẩn đoán và đào nhãn là cùng một thao tác, chỉ khác cách đọc kết quả.

Ô *đường tắt* là ca đáng chú ý: model lấy đúng D mà không đường liền mạch nào qua được kiểm chứng — nó đang trả lời bằng tín hiệu lan man chứ không bằng chuỗi suy luận. Đây là một chỉ số kiểm toán nên báo cáo riêng,

$$\text{tỷ lệ đường tắt} \;=\; \frac{\lvert\{\,\text{rank tốt},\ \text{suff} < \theta_s\,\}\rvert}{\lvert\{\,\text{rank tốt}\,\}\rvert},$$

đo trên mẫu 5–10% ví dụ rank tốt, vì bộ lọc rẻ-trước-đắt-sau bình thường bỏ qua chúng.

Khi rơi vào ô *lỗi đồ thị*, đọc thêm lý do dừng của greedy (§4.11) để biết sửa ở đâu: gain biên dưới ε nghĩa là bằng chứng phân tán — đọc kèm $\sigma(D)$ để biết phân tán kiểu gì: $\sigma(D)$ cao là nhiều nguồn thật mỗi nguồn nhỏ (bằng chứng vững), $\sigma(D) = 1$ là một nguồn duy nhất và mong manh (đáng ngờ); chạm $n_{\max}$ nghĩa là cần hơn $n_{\max}$ lối độc lập, đồ thị dư thừa cao.

"Xếp sai" nghĩa là gì

$$\text{rank}_{\text{GFM}}(D) = \left|\{\, v : \text{type}(v) = \text{document},\ p(v) > p(v_D) \,\}\right| + 1$$

Vượt ngưỡng nghĩa là **lúc chạy thật, LLM sẽ không nhìn thấy đoạn này** — dù tầng A đã xác nhận nó cần.

k_{top} phải thay đổi theo từng ví dụ

$$k_{\text{top}}(q) = \lceil \beta\, |T(q)| \rceil, \qquad \beta \approx 1{,}5$$

Để k_{top} là hằng số thì hỏng theo cách khó thấy. Với $k_{\text{top}} = 5$ và $|T| = 8$, ít nhất ba đích **luôn luôn** bị gắn "lỗi model" bất kể model tốt đến đâu; ba ví dụ đó không bao giờ rời khỏi S_k, nên $|S_{k+1}| \approx |S_k|$ ngay từ mốc 2 và tiêu chí dừng §6.5 báo bão hòa giả. $k_{\text{top}} = |T|$ là đòi hỏi chặt nhất mà vẫn luôn thỏa mãn được; $\beta \approx 1{,}5$ chỉ nới chút cho thực dụng. Hai ngưỡng, hai mục đích:

```
bộ lọc train        rank <= k_top(q)    "model đã xếp tốt nhất có thể chưa?"
báo cáo hiệu năng   rank <= k_infer     "đoạn này có thật sự được truy hồi không?"
```

Thứ tự: rẻ trước, đắt sau

```
bước A   rẻ     1 lượt forward mỗi ví dụ  ->  rank_GFM(D)
                loại ngay ví dụ model đã xếp đúng
                còn lại  F = { model đang xếp sai }

bước B   đắt    chạy tầng B chỉ trên F
                IG -> QHĐ -> greedy -> suff
                tách F theo bảng chẩn đoán ở trên
```

Chi phí IG chỉ trả cho $|F|$ ví dụ thay vì cả tập.

Vì sao nó đứng sau $\mathcal{L}_{\text{node}}$ nhưng trước $\mathcal{L}_{\text{hop}}$

Hai đại lượng của bảng đều vô nghĩa trên model chưa train: rank thì mọi đích đều tệ, suff thì IG chỉ là nhiễu. Chạy sớm là ra *100% lỗi model* — không phân biệt được gì. Mà mục đích của bảng chính là phân biệt **model dở** với **đồ thị thiếu**; muốn tách được thì model phải được cho cơ hội công bằng trước đã.

Bảng là cái cổng quyết định có nên bước qua vạch "đổi kiến trúc" không. Nếu phần lớn rơi vào lỗi đồ thị hoặc lỗi cấu trúc thì dừng lại sửa graph, đừng thêm loss. Với QuadGraph, "không có đường trong L hop" thường nghĩa là thiếu cạnh liên tầng (entity→document).

Khi lý do dừng của greedy vẫn chưa tách bạch, dùng thêm **độ dài đường** làm gợi ý: đường dài sát L thì nghiêng về đồ thị thiếu lối; có đường ngắn mà suff vẫn thấp thì nghiêng về bằng chứng phân tán.

Đọc phân bố qua các mốc

```
             mốc 1    mốc 2    mốc 3
ổn            45%      68%      74%
lỗi model     38%      17%      12%     <- S_k, co lại = đang ăn
lỗi đồ thị    14%      13%      13%     <- không co = phải sửa graph
lỗi cấu trúc   3%       2%       1%
```

Hàng thứ hai chính là thứ mà tiêu chí dừng §6.5 đọc.

6.2 Trọng số theo quan hệ phụ thuộc

Nhãn các đích không độc lập: model trượt D_7 thì không đời nào với tới D_{15}. BCE phẳng giả định độc lập nên sai ở đây.

$$w(D) = 1 + \left|\{\, D' \in T : D \text{ nằm trên } \Pi_{D'} \,\}\right|$$

Ví dụ: $w(D_7) = 2$, $w(D_{15}) = 1$. Dùng kèm curriculum — train đích depth nhỏ trước (depth cũng chỉ có từ mốc 1).

$w(D)$ **chưa có ở mốc 0.** Công thức trên cần Π, mà Π chỉ xuất hiện từ mốc 1 (§6.5). Ở mốc 0 dùng $w(D) = 1$ cho mọi đích, tức BCE phẳng. Cách khác: suy quan hệ phụ thuộc ngay ở tầng A — $v(\{D_{15}\}) \approx 0$ nhưng $v(\{D_7, D_{15}\})$ cao thì D_{15} phụ thuộc D_7 — khi đó $w(D)$ độc lập với model và cache được cùng T, đổi lại tốn thêm vài lượt LLM cho các cặp đích.

6.3 Ba hàm mất mát

$$\mathcal{L}_{\text{node}} = -\sum_{D \in T} w(D) \log p(D) \;-\; \sum_{v \in V_{\text{neg}}} \log\big(1 - p(v)\big)$$

$$\mathcal{L}_{\text{hop}} = \sum_{l=1}^{L} \sum_{v \in V_{\text{hop}}(l)} \text{BCE}\Big(p^{(l)}(v),\; \mathbf{1}\big[\exists\, \pi \in \Pi : (v, l) \in \pi\big]\Big)$$

$$\mathcal{L}_{\text{edge}} = \sum_{e,l} \text{BCE}\left(\hat{g}_e^{(l)},\; \widehat{w}_e^{(l)}\right)$$

$$\mathcal{L} = \mathcal{L}_{\text{node}} + \lambda_1 \mathcal{L}_{\text{hop}} + \lambda_2 \mathcal{L}_{\text{edge}}$$

$\mathcal{L}_{\text{node}}$ thay nhãn cũ "đoạn có chứa chuỗi đáp án" — nhãn cũ đầy dương tính giả. $\mathcal{L}_{\text{hop}}$ cần thêm predictor nhỏ $p^{(l)}$ ở mỗi tầng; đây là giám sát trung gian, vá chỗ credit assignment bị đứt qua L bước (§4 của phụ lục *Những chỗ dễ sai*). $\mathcal{L}_{\text{edge}}$ dạy trực tiếp cho gate phụ thuộc truy vấn ở §4.2 chế độ 2, với $\widehat{w}$ là IG chuẩn hóa về $[0, 1]$.

$\mathcal{L}_{\text{hop}}$ phải lấy mẫu âm, không cộng trên mọi node. Cộng hết thì tỷ lệ dương/âm vô vọng: ô dương là node nằm trên Π đúng tầng, nhiều nhất $n_{\max} \times L$ tức cỡ 10 ô; ô âm là $|V| \times L$ tức cỡ 10^7. Ở tỷ lệ $1 : 10^6$ thì nghiệm tầm thường "luôn đoán 0" đã gần tối ưu và loss gần như không nhúc nhích. $\mathcal{L}_{\text{node}}$ đã tránh điều này bằng V_{neg}; $\mathcal{L}_{\text{hop}}$ gặp đúng vấn đề đó, còn nặng hơn vì nhân thêm L. Nên hạn chế miền cộng:

```
V_hop(l)  =  { v : (v,l)  thuộc Pi }          dương -- node trên đường, đúng tầng
          U  { v : (v,l') thuộc Pi,  l' != l } âm khó -- đúng node, sai tầng
          U  V_neg                            âm thường -- dùng lại mẫu của L_node
```

Nhóm giữa mới là nhóm đáng giá: nó dạy **thời điểm**, thứ mà nhãn chỉ ở tầng cuối không nói được. Ví dụ ngay dưới có đúng một ô như vậy — $(D_7, \text{tầng } 1) = 0$ — và tài liệu vốn đã chỉ ra nó là ô đáng chú ý nhất. Lấy mẫu kiểu này giữ nguyên mọi ô mà ví dụ đó nêu.

Nhãn cho từng loại node

QuadGraph có bốn loại node, mà T chỉ nói về đơn vị văn bản. Chia vai:

	vào prompt?	nguồn nhãn	loss
`document`	có	T (tầng A)	$\mathcal{L}_{\text{node}}$
`community`	có	T (tầng A)	$\mathcal{L}_{\text{node}}$
`entity`	không	Π (tầng B)	$\mathcal{L}_{\text{hop}}$
`attribute`	không	Π (tầng B)	$\mathcal{L}_{\text{hop}}$

Quy tắc chốt: **thứ gì vào prompt thì phải từng là người chơi trong trò chơi §3.3.** Nhét vào prompt một thứ tầng A chưa bao giờ chấm thì T không phủ được nó, $\mathcal{L}_{\text{node}}$ không giám sát nó, và chuỗi quy gán đứt.

`entity` và `attribute` không bao giờ vào prompt. Chúng là chất nền để lan truyền và là vật liệu cho lời giải thích — đường Phim-X $\to$ Ông-Z $\to D$ chạy xuyên qua chúng — nên được giám sát gián tiếp qua Π, không qua T. Kéo theo k_{infer} là ngưỡng của *riêng* `document` + `community`.

$p^{(l)}$ dùng $4L$ head, một head cho mỗi cặp (tầng GNN $\times$ loại node). Khớp với $\text{Predictor}_{\text{type}(v)}$ mà G-reasoner vốn đã có, và cần thiết vì tần suất "nằm trên đường" của `document` khác hẳn của `entity`.

V_{neg} lấy trong loại `document`, và tầng A cho sẵn hai nguồn với chất lượng khác hẳn nhau:

```
beta_i < 0                negative tốt nhất -- đoạn đẩy ngược câu trả lời (§3.7)
không nằm trong MWC nào   negative thật -- vô dụng ở mọi tập đủ tối tiểu

nằm trong một số MWC nhưng không trong S*
                          không phải negative. Đoạn này có ích, chỉ là lời giải
                          thích khác đã dùng nó. Phạt nó là dạy retriever tránh
                          một đoạn đúng.
```

Đây là chỗ bản MWC đọc sạch hơn bản Shapley: "không nằm trong MWC nào" là một phát biểu dứt khoát, còn "thuộc cụm bị cắt" thì phụ thuộc ngưỡng cắt cụm. Lấy mẫu đều trên toàn đồ thị cũng sai — phần lớn rơi vào `entity/attribute`, và loss chỉ dạy được "đây không phải node loại document", tức phân biệt *loại* chứ không phải *nội dung*.

Ví dụ số — cùng một đồ thị cho cả hai loss

```
   X --e1--> C --e2--> D7          đường đúng,  V_q={X}, L=3
    \                   ^
     \--e3--> J --e4----+          nhánh lạc

Pi_D7 = [ (e1,tang1), (e2,tang2) ]   ->  o: (X,0) (C,1) (D7,2) (D7,3)
```

$\mathcal{L}_{\text{hop}}$ — nhãn đọc thẳng từ Π, $y = 1$ nếu ô nằm trên đường:

```
nhãn y^(l)(v)       tầng1  tầng2  tầng3     model đang đoán p^(l)(v)
  X                   0      0      0        0.30   0.20   0.15
  C                   1      0      0        0.45   0.30   0.25
  J                   0      0      0        0.40   0.35   0.30
  D7                  0      1      1        0.10   0.35   0.55
```

```
ba số hạng nặng nhất:
  (C,  tầng1)  y=1 p=0.45  ->  -log 0.45 = 0.799   "nâng C ở tầng 1"
  (D7, tầng2)  y=1 p=0.35  ->  -log 0.35 = 1.050   "nâng D7 ở tầng 2"
  (D7, tầng3)  y=1 p=0.55  ->  -log 0.55 = 0.598   "giữ D7 tới cuối"
                                       L_hop = 5.236
```

Chú ý ô $(D_7, \text{tầng } 1) = 0$: D_7 không được phép sáng ở tầng 1 vì tin chưa kịp tới. Nhãn chỉ ở tầng cuối không nói được điều này.

$\mathcal{L}_{\text{edge}}$ — nhãn là IG chuẩn hóa $\widehat{w} = \text{IG}/\max \text{IG}$:

```
w^_e^(l)   tầng1  tầng2  tầng3     g^ hiện tại   tầng1  tầng2  tầng3
e1 X->C     1.00   0.06   0.03                    0.62   0.55   0.50
e2 C->D7    0.00   0.94   0.10                    0.58   0.60   0.52
e3 X->J     0.06   0.00   0.00                    0.65   0.50   0.48
e4 J->D7    0.00   0.03   0.00                    0.55   0.52   0.50

ba số hạng nặng nhất:
  e3 tầng1   w^=0.06 g^=0.65  ->  1.013   "đóng X->J, nó vô dụng"
  e2 tầng1   w^=0.00 g^=0.58  ->  0.868   "đóng C->D7 Ở tầng 1"
  e2 tầng2   w^=0.94 g^=0.60  ->  0.535   "nhưng mở C->D7 ở tầng 2"
                                       L_edge = 8.671
```

Hai dòng `e2` là cả bài học: cùng một cạnh, đóng ở tầng 1 và mở ở tầng 2. Gate chung cho cả L tầng không diễn đạt nổi câu đó — nó chỉ học được một con số trung bình.

> **BCE với nhãn mềm không bao giờ về 0.** Sàn của nó là entropy của chính nhãn, $\sum H(\widehat{w}) = 1{,}276$ trong ví dụ này. Thấy $\mathcal{L}_{\text{edge}}$ dừng ở $1{,}28$ là đã hội tụ, không phải kẹt.

$\mathcal{L}_{\text{hop}}$ giám sát node theo tầng ("ai sáng lúc nào", kết quả); $\mathcal{L}_{\text{edge}}$ giám sát cạnh theo tầng ("lối nào mở lúc nào", cơ chế). Không thừa nhau: $\mathcal{L}_{\text{hop}}$ nói *C phải sáng ở tầng 1* nhưng không nói đi qua cạnh nào để làm C sáng. Điểm chung là cả hai dạy *thời điểm* — đúng thứ mà $\mathcal{L}_{\text{node}}$, chỉ nhìn $p(v)$ ở tầng cuối, mù hoàn toàn.

6.4 Bộ lọc cho giám sát đường đi

Chỉ áp $\mathcal{L}_{\text{hop}}$ và $\mathcal{L}_{\text{edge}}$ lên ví dụ thỏa cả ba:

$D \in T$	tầng A nói cần
$\text{rank}_{\text{GFM}}(D) > k_{\text{top}}(q)$	model đang xếp sai (*đã đúng thì dạy số 0*)
$\text{suff}(\Pi_D) \geq \theta_s$	đường đã kiểm chứng (*thầy sai thì đừng dạy*)

Lý do điều kiện thứ hai: T đến từ LLM nên là thông tin mới thật sự; còn Π đến từ chính GFM nên phần lớn là tự chưng cất. Tập con mang thông tin mới là những ca model đang sai — ở đó IG chỉ ra các cạnh *lẽ ra* phải dùng để nâng $p(D)$ mà model hiện chưa dùng.

Câu trên **chỉ đúng khi đo với** $g^* = 1$ (§4.4). Nếu đo bằng $\hat{g}$ thì cạnh model chưa dùng nhận IG ≈ 0 và không bao giờ vào Π; $\mathcal{L}_{\text{edge}}$ khi đó chỉ dạy gate củng cố đúng cái nó đang làm.

6.5 Lịch đào nhãn và vòng lặp khởi động

Nhãn không được tính trong lúc train. Chúng do một job riêng sinh ra, chạy *giữa* các đợt train, với trọng số GFM đóng băng, ghi ra file; lúc train chỉ đọc file đó.

Nhãn nào có từ lúc nào

Nhãn	Nguồn	Phụ thuộc model?	Có từ
T, neg	tầng A — MWC trên LLM	không	mốc 0, cache vĩnh viễn
Π	tầng B — IG + QHĐ	có	mốc 1, đào lại từng mốc
$\widehat{w}$	tầng B — IG chuẩn hóa	có	mốc 1, đào lại từng mốc
depth(D)	tầng B — bảng QHĐ §4.8	có	mốc 1
$w(D)$	suy từ Π	có	mốc 1 (mốc 0 dùng $w = 1$)

Bốn hàng dưới **đều cần tầng B**, mà tầng B cần GFM — nên không thứ nào có ở mốc 0. Kéo theo: curriculum theo depth (§6.2) cũng chỉ bật được từ mốc 1. Π và $\widehat{w}$ ra từ **cùng một bảng** $\mathrm{IG}_e^{(l)}$, chỉ khác cách đọc: qua QHĐ + greedy thì ra Π_D và depth(D), qua chuẩn hóa về $[0, 1]$ thì ra $\widehat{w}_e^{(l)}$.

Vì sao Π và $\widehat{w}$ không thể có từ đầu

$\mathrm{IG}_e^{(l)} = \hat{g}_e^{(l)} \cdot \partial F_B / \partial g_e^{(l)}$ — đạo hàm của model, phụ thuộc trọng số hiện tại. Để biết *"model lẽ ra phải đi lối nào"* thì phải hỏi chính model, mà câu trả lời đổi khi model đổi. Cách gỡ: đóng băng model, hỏi một lần, train một hồi trên câu trả lời đó, rồi hỏi lại. T thì khác hẳn — nó đến từ LLM sinh, không liên quan GFM, nên đào một lần là xong vĩnh viễn.

Vì sao không đào lại mỗi epoch

```
đào một ví dụ     M forward              =  20
                + M*|T| backward         = 120 bwd  ~ 240 forward-tương-đương
                + kiểm chứng             ~  13
                                           -----
                                           ~ 273 lượt forward-tương-đương

train một ví dụ   1 fwd + 1 bwd          ~   3 lượt
```

Đào một ví dụ tốn bằng ~ 91 **bước train** trên chính nó. Cứu cánh là bộ lọc §6.4: chỉ đào cho ví dụ model **đang sai**, và tập đó co lại theo thời gian.

Job đào nhãn làm gì

```
job  mine_stage_B( checkpoint, tập ví dụ ):

    tải trọng số GFM từ checkpoint, đóng băng -- không cập nhật gì

    cho mỗi ví dụ q:
        1 forward  ->  p(v) mọi node  ->  rank_GFM(D) cho từng D thuộc T
        nếu mọi D đều có rank <= k_top(q):
            bỏ qua ví dụ này     <- model đã đúng, Pi sẽ chỉ là tự chưng cất
        ngược lại, với mỗi D còn lại:
            M forward + M backward trên gate   ->  bảng IG_e^(l)
            QHĐ trên bảng đó                   ->  best[], back[]
            depth(D) = cột đầu tiên hết -inf
            truy ngược + greedy + suff §4.10   ->  Pi_D
            IG / max IG                        ->  what
        w(D) = 1 + |{ D' : D nằm trên Pi_D' }|

    ghi file nhãn
```

Ba mốc, và một ví dụ đi qua chúng

```
mốc 0   một lần, vĩnh viễn -- chỉ chạy LLM, không đụng GFM
        { q, V_q, T = [D7], neg = [...], w = {D7: 1} }
        chưa có Pi, chưa có what, chưa có depth
        train  L_node  tới hội tụ

mốc 1   chạy job trên, gate = 1 (chế độ 1)
        { ... cộng thêm:
          rank_lúc_đào = {D7: 23},        ngoài k_top -> ví dụ này được giữ
          depth = {D7: 2},
          Pi = {D7: [ [(X->F2,t1), (F2->D7,t2)],
                      [(Y->F2,t1), (F2->D7,t2)] ]},
          what = {(X->F2,t1): 0.67, (Y->F2,t1): 0.58, (F2->D7,t2): 1.00},
          suff = {D7: 0.97},  w = {D7: 1} }

        Pi dùng ngay      ->  L_node + lambda_1 * L_hop
        what nằm trong file nhưng chưa dùng -- chưa có ghat để dạy

mốc 2   thêm module gate (chế độ 2) vào model, lọc lại S_2, chạy lại job
        cách đo không đổi -- vẫn g* = 1 như mốc 1 (§4.4)
        Pi và what bị ghi đè bằng bản sắc hơn
        what giờ mới dùng được  ->  + lambda_2 * L_edge

mốc 3   tùy chọn, lặp lại mốc 2
```

Ba mốc này chính là lời giải cho bài toán con gà quả trứng ở trên: mốc 1 chạy IG với gate hằng bằng 1, mốc 2 chạy lại với gate đã học. Giữa hai mốc thì **nhãn đóng băng** — train nhiều epoch trên cùng một Π, chấp nhận được vì Π mã hóa "lẽ ra phải đi lối nào", thứ không đổi nhanh theo điểm số của model. Chi phí một lần đào ở mốc 1 là $|S_1|/|D|$ phần chi phí đầy đủ; nếu 20% ví dụ sai thì $0{,}2 \times 91 \approx 18$ epoch tính toán.

Tiêu chí dừng

DỪNG KHI $\quad |S_{k+1}| \approx |S_k| \quad$ — tập lọc ngừng co lại

Tiêu chí này **chỉ đúng khi k_{top} đặt theo từng ví dụ** (§6.1). Với k_{top} hằng số, những ví dụ có $|T|$ lớn kẹt lại trong S_k vĩnh viễn và làm tiêu chí báo bão hòa giả.

Tập lọc không nhỏ đi nữa nghĩa là $\mathcal{L}_{\text{hop}}$ và $\mathcal{L}_{\text{edge}}$ đã hết chỗ sửa: đào thêm chỉ lấy về nhãn cho những ca model sẽ tiếp tục sai vì lý do khác, thường là thiếu cạnh trong đồ thị — hàng thứ ba của bảng §6.1. Thực tế 2–3 mốc là bão hòa; nếu mốc 3 vẫn co mạnh thì nhiều khả năng đang học nhiễu của corpus — giảm λ_2 hoặc đa dạng hóa corpus đào nhãn.

6.6 Vòng lặp ngược — dùng đích để tìm ứng viên mới

Bình thường seed là V_q; vòng ngược lấy seed là T. Phần sắc nhất nằm ở phép trừ tập hợp:

$$\text{ứng viên đáng giá} = R(T) \setminus R(V_q)$$

Tức đoạn mà bằng chứng tốt dẫn tới, nhưng câu hỏi không dẫn tới. Nếu nằm trong cả hai thì retriever lẽ ra đã tìm được rồi. Phần hiệu mới là chỗ exposure bias đang trốn.

```
vòng n:    pool = bm25 + GFM + ngẫu nhiên              -> tầng A -> T_n
vòng n+1:  pool = pool cũ + top20( R(T_n) \ R(V_q) )   -> tầng A -> T_{n+1}
```

Chi phí phía đồ thị gần bằng 0; chi phí thật là chạy lại tầng A trên pool đã mở rộng — f_θ dùng lại được nhưng vòng 0 và vòng 3 phải chạy lại, nên giới hạn 10–20 node. Hai vòng là bão hòa.

6.7 Kỳ vọng thực tế

Single-hop gần như không cải thiện — nhãn node đã nói đủ. Multi-hop là chỗ nên ăn. G-reasoner đã đạt EM $81{,}1$ trên HotpotQA (gần trần), nên đo trên bộ khó hơn (MuSiQue, 2Wiki). Đây là giả thuyết có cơ sở, chưa phải kết quả đã chứng minh.

7 Chi phí

7.1 Một truy vấn, pipeline đầy đủ

$$\text{Tầng A} = \underbrace{n+3}_{\text{vòng 0}} + \underbrace{(r-1)\cdot m}_{\text{vòng 1}} + \underbrace{n_{\text{act}}}_{\text{vòng 2}} + \underbrace{|S^*| + 2^{|S^*|} - 2}_{\text{vòng 3, model thật}} \text{ lượt LLM}$$

$$\text{Tầng B} = \underbrace{M}_{\text{forward}} + \underbrace{M \cdot |T|}_{\text{backward}} + \Big(1 + 2\textstyle\sum_D |\Pi_D|\Big) \text{ lượt GFM}$$

Vòng 0 gồm $v(\emptyset)$, $v(C)$, $v(R)$ và mọi singleton (§3.4); vòng 3 xác nhận đủ **và** quét hết tập con (§3.10). Quy ra token, với $n = 40$, $C = 6$, $m = 12$, $r = 3$, $|S^*| = 3$:

```
vòng 0    v(rỗng) 500 + v(R) 3.500 + v(C) 10.500 + 40 singleton 30.000   44.500
vòng 1    (r-1)*m = 24 lượt                                               27.000
vòng 2    ~30 lượt                                                        37.000
vòng 3    1 + 6 lượt                                                       6.500
                                                                        ---------
                                                                          115.000 token
                                                                  ~ $0,35 ở $3/triệu
```

Tầng B gần như miễn phí bên cạnh: 20 fwd $+ 120$ bwd $+ 13$ fwd trên một mô hình $34M$ tham số. Hai nửa chênh khoảng **140 lần**. Hệ quả: **cache tập đích T lại.** Nhãn $(q \to T)$ không phụ thuộc kiến trúc GFM — đào một lần, dùng lại cho mọi lần train, mọi thay đổi đồ thị, mọi giá trị L.

7.2 Quy mô thật — hai pha

Con số $\$0{,}35$ ở trên nhân với số truy vấn thật thì không trả nổi, nên §3.11 tách hai pha.

	truy vấn	\$/truy vấn	tổng
pha 1, pipeline đầy đủ	$18.200 \to 10.000$	\$0,38	\$3.850
pha 2, đường rẻ	$200.000 \to 110.000$	\$0,037	\$7.500
cứu, có phân tầng	3.000	\$0,34	\$1.000
hiệu chuẩn HotpotQA	500	\$0,38	\$190
tổng			**\$12.540**
chạy pipeline đầy đủ cho cả 200.000			\$76.000

Cột trái ghi *nạp vào → chạy đủ*: cửa thoát `A0/C0` bỏ $\sim$45% truy vấn với giá 400 token — các ca mà bảng phủ nói pool *không chứa* tập đủ. Chúng không bị xóa: chúng là cờ "pool thiếu", nguồn nuôi ô «lỗi đồ thị» §6.1 và vòng lặp ngược §6.6.

Khoảng **6 lần rẻ hơn**, cho $\sim$104k nhãn *đáng tin*. So với bản trước ($\$19.500$ cho $\sim$188k nhãn) thì rẻ hơn *và* ít nhãn hơn — đó là tiến bộ: 84.000 nhãn chênh lệch là các truy vấn mà pool không chứa tập đủ, nên nhãn sinh ra từ chúng hoặc rỗng hoặc là mồi tham số. Phần tiết kiệm đến từ việc bỏ vòng 0 ở pha 2 — 40 lượt singleton (30.000 token) thay bằng g_θ, cộng vòng 1–2 (64.000). *Không* từ việc cắt verification, và *không* từ việc bỏ $v(C)$: $v(C)$ chỉ là 10.500 trong 106.000 token tiết kiệm được, tức 10%.

```
C1  v(rỗng)                   500 token    2%
C3  v(R)                    3.500 token   16%
C4  v(C)                   10.500 token   48%
C6  verification, kỳ vọng   7.400 token   34%
                           -------------
                           21.900 token
```

$v(C)$ thành phần nặng nhất, và đó là **đúng giá của một cái trần chính xác**: bù nó bằng một hằng số hiệu chuẩn thì $\sim$10% truy vấn bị trần thấp, sinh nhãn sai mà verification không bắt được (§3.11). Còn cắt C6 thì T không tối tiểu vẫn được xuất, và tầng B học hướng thừa.

7.3 Sổ dữ liệu

Chi phí trên mua về hai thứ cùng lúc: nhãn cho tầng B, và dữ liệu train cho g_θ.

```
nguồn                            truy vấn   nhãn/truy vấn    tổng
pha 1, bốn vòng                   10.000         111        1.110.000
verification đường rẻ            188.000          10        1.880.000
tỉa, |S*| = 6..10                  8.000          13          104.000
cứu -> pipeline đầy đủ             7.400         111          821.000
ca trượt bị bỏ, không cứu         13.600           5           68.000
                                                            -----------
                                                            3.983.000
```

Pha 2 **sinh ra nhiều dữ liệu train hơn cả pha 1**, và chúng nằm đúng vùng phân bố mà beam tra tới còn greedy chưa đi qua. Đó là lý do chạy theo lô (§3.11): mỗi lô nạp lại thì p tăng và lô sau rẻ hơn.

7.4 So với bản Shapley

Bản Shapley ở file riêng tốn $\sim$128 lượt mỗi truy vấn — xấp xỉ bằng. Giá như nhau nhưng đổi được cả họ MWC, chỉ số cấu trúc và độ tin cậy thay vì một tập T duy nhất. Cái mất là completeness (§8).

Điều bản Shapley *không* làm được là chế độ hai pha: điểm φ_c của nó định nghĩa trên toàn bộ pool nên không có cách nào xấp xỉ bằng một hàm tập rồi xác nhận lại bằng vài lượt. MWC thì có, vì "đủ và tối tiểu" là một vị từ kiểm được trên một tập con cụ thể.

7.5 Quy tắc chọn IG hay ablation

$$\boxed{\text{Quy tắc}\quad \text{IG rẻ hơn ablation} \iff N > 3M}$$

với N là số đơn vị cần chấm, *điều kiện*: prompt hoặc đồ thị không phình theo N. Hệ số 3 vì backward $\approx 2\times$ forward.

cạnh đồ thị (hàng nghìn, đồ thị cố định)	IG
token trong đoạn (hàng trăm, prompt cố định)	IG
đoạn trong pool (prompt dài theo số đoạn)	ablation / tập đủ tối tiểu

Chứng minh dòng cuối — khi prompt phình theo K, gọi c là chi phí một forward trên prompt ngắn:

$$\text{standalone} = K \cdot c, \qquad \text{IG} = M \cdot 3 \cdot \underbrace{K \cdot c}_{\text{prompt dài}}, \qquad \frac{\text{IG}}{\text{standalone}} = 3M$$

K triệt tiêu hoàn toàn — không có điểm hòa vốn. Cảm giác "IG chấm một lượt được hết K đoạn" là ảo giác, vì mỗi lượt phải nuốt cả K đoạn.

8 Bất biến phải kiểm

$$
\begin{array}{lll}
\text{tầng A} & (1) & v(S^*) \;\geq\; \tau_q \qquad\qquad\qquad \text{đo bằng model thật} \\
& (2) & v(S^* \setminus \{c\}) \;<\; \tau_q \qquad \text{mọi } c \in S^* \\
& (3) & \{\, i : \beta_i < 0 \,\} \cap S^* = \emptyset \\
& (4) & \textstyle\sum_i \text{credit}(c_i) = 1 \\
\text{tầng B} & (5) & \displaystyle\sum_{e,l} \text{IG}_e^{(l)} = F_B(g^*) - F_B(0) \\
& (6) & \text{đường } h \text{ hop giá trị } c \implies \text{mỗi hop} \approx c/h
\end{array}
$$

(1)–(3) phải chạy bằng **model thật**, không phải f_θ — đó là toàn bộ ý nghĩa của chúng. (4) đúng theo cấu trúc. (5) chỉ đúng tới sai số rời rạc hóa của M — lệch quá 5% thì tăng M. (6) là chẩn đoán: lệch nhiều nghĩa là bạn đang nhìn hai đường chứ không phải một.

Bộ của tầng A giờ yếu hơn bản Shapley. Đẳng thức $\sum \phi_c = F(E) - F(E')$ là một ràng buộc chính xác trên *toàn bộ* phép tính, bắt được mọi lỗi cài đặt ở khâu quy gán.

Chuẩn ngoài — hiệu chuẩn trên HotpotQA. Mọi phép kiểm ở trên là **nội bộ**: $v(T) \geq \tau_q$ do chính v đo, τ_q do chính v_{ref} đặt, adq do chính model NLI của ta phán. Không có gì bên ngoài nói "T này đúng là nguồn".

HotpotQA có thứ đó: nó gán sẵn **2 đoạn chống đỡ** và tối đa **5 câu chống đỡ** cho mỗi câu hỏi — nhãn quy gán do người gán, chuẩn ngoài duy nhất sẵn có.

```
lấy ~500 truy vấn HotpotQA -> chạy TOÀN BỘ tầng A -> so T với 2 đoạn đã gán
giá:  500 x $0,38  =  $190

1  tỷ lệ ca mỗi tham số THẬT    T chứa đoạn không nằm trong nguồn vàng
2  model NLI có đáng tin không   adq có đồng ý với nhãn người không
3  precision/recall của T        tầng A đúng bao nhiêu phần so với chuẩn ngoài
```

Con số 2 quan trọng nhất: đem adq đi loại nhãn trên 200.000 truy vấn mà *chưa bao giờ biết nó sai bao nhiêu* thì chỉ là đổi một lỗi không biết sang một lỗi khác không biết. Hai lỗi cụ thể phải đo: *phân rã sót một hop* (bảng phủ thiếu cột, T được nhận khi còn thiếu bằng chứng) và *NLI "neutral" giả* (thoát sớm oan, hoặc loại oan T tốt).

Một chỗ phải cẩn thận: HotpotQA gán nguồn cho bối cảnh 10 đoạn có nhiễu của riêng nó, còn pool của ta dựng từ 3 nguồn trên corpus riêng với cách cắt chunk của ta. Cần bước **khớp chunk của ta với đoạn của họ** trước khi so — làm được, nhưng là một bước thật.

MuSiQue bù chỗ khác: nó không gán câu chống đỡ, nhưng gán **phân rã thành câu hỏi 1-hop kèm đáp án trung gian**, tức cho sẵn bước 1 của adq (§3.2) — dùng nó để kiểm chất lượng của LLM chẻ câu hỏi.

Phép kiểm (1)–(5) chỉ xác nhận T, không xác nhận cách ta tìm ra nó. Đó là cái giá của việc bỏ completeness, và nên nói thẳng chứ đừng giấu. Bù lại bằng đối chứng: chạy bản Shapley ở tài liệu riêng trên một mẫu nhỏ, kiểm xem S^* có nằm trong nhóm đoạn mà Shapley chấm cao không.

Không có bất biến nào nối tầng A với tầng B.

9 Thứ tự triển khai

1. Tầng A đầy đủ, xuất T — *rẻ nhất, dùng được ngay*
2. Đổi nhãn train GFM sang T, với $w(D) = 1$ — *chắc ăn nhất*
   ($w(D)$ thật cần Π, chỉ có từ mốc 1)
3. Chạy bảng chẩn đoán (§6.1) — *lỗi ở model hay ở đồ thị?*
4. Nếu lỗi model → thêm $\mathcal{L}_{\text{hop}}$ — *cách dễ trước*
5. Nếu vẫn thiếu → gate query-aware + $\mathcal{L}_{\text{edge}}$ — *đổi kiến trúc, sau cùng*
6. Vòng lặp ngược (§6.6) — *khi nghi exposure bias*

```
bước 1-2  =  mốc 0  (§6.5)      bước 3-4  =  mốc 1      bước 5  =  mốc 2
```

Đừng làm bước 5 trước bước 3. Nếu phần lớn lỗi là thiếu cạnh trong đồ thị, mọi cải tiến loss đều vô nghĩa.

Tham chiếu

[1] L. S. Shapley. *A value for n-person games.* Contributions to the Theory of Games II, 1953.

[2] M. Sundararajan, A. Taly, Q. Yan. *Axiomatic Attribution for Deep Networks.* ICML 2017.

[3] G. Owen. *Multilinear extensions of games.* Management Science 18(5), 1972.

[4] J. Deegan, E. W. Packel. *A new index of power for simple n-person games.* International Journal of Game Theory 7(2), 1978. — credit chia đều trong mỗi liên minh thắng tối tiểu.

[5] J. F. Banzhaf III. *Weighted voting doesn't work: A mathematical analysis.* Rutgers Law Review 19, 1965.

[6] M. J. Holler, E. W. Packel. *Power, luck and the right index.* Journal of Economics 43(1), 1983. — Public Good Index.

[7] G. Izacard, E. Grave. *Distilling Knowledge from Reader to Retriever for Question Answering.* ICLR 2021.

[8] G. Izacard và cộng sự. *Atlas: Few-shot Learning with Retrieval Augmented Language Models.* JMLR 24, 2023. Nguồn của ADist / PDist / LOOP / EMDR^2.

[9] L. Luo, Z. Zhao, R. Haffari, D. Phung, C. Gong, S. Pan. *GFM-RAG: Graph Foundation Model for Retrieval Augmented Generation.* arXiv:2502.01113, NeurIPS 2025. https://arxiv.org/abs/2502.01113

[10] L. Luo, S. Pan và cộng sự. *G-reasoner: Foundation Models for Unified Reasoning over Graph-structured Knowledge.* arXiv:2509.24276, 2025. https://arxiv.org/abs/2509.24276 — kiến trúc mà tài liệu này bám theo.

[11] R. Ying và cộng sự. *GNNExplainer: Generating Explanations for Graph Neural Networks.* NeurIPS 2019.

[12] M. S. Schlichtkrull, N. De Cao, I. Titov. *Interpreting Graph Neural Networks for NLP With Differentiable Edge Masking.* ICLR 2021.